

NAT——网络地址转换

IP 地址非常匮乏。一个 ISP 或许有一个/16 地址块，给它的 65 534 个可用主机编号。如果有比这更多的客户，那就有问题了。

这种匮乏导致了一些保守使用 IP 地址的技术。其中一种方法是为一台连在网上并使用网络的计算机动态分配一个 IP 地址，而且在该主机不活跃时收回分配给它的 IP 地址；然后该 IP 地址可以被分配给另一台活跃的计算机。以这种方式，一个/16 地址可以处理多达 65 534 个活跃用户。

这种动态分配策略在某些情况下运行良好，例如，那些可能暂时缺席或断电的拨号上网、移动和其他计算机。但是，它并不能很好地为企业客户工作。企业中的许多 PC 是要持续打开的。有些员工的机器在晚上需要做备份，而有些服务器可能需要在顷刻间服务于远程客户的请求。这些企业通常有一条总是提供与其余 Internet 连通的接入线路。

渐渐地，因为没有连接费用（仅仅是包月收费），这种情况出现在订购了 ADSL 或线路上 Internet 服务的家庭用户。这些用户中的很多人在家里有两个或更多台计算机，通常每个家庭成员都有一台，而且他们都希望所有的时间都在线。相应的解决方案是通过局域网把所有的计算机连接到一个家庭网络，并且在家庭网络上放置一台（无线）路由器；然后路由器连接到 ISP。从 ISP 的角度来看，家庭网络现在就像一个拥有少数计算机的小型企业。用我们迄今为止所看到的技术，每台计算机必须整天都有它自己的 IP 地址。对于有成千上万客户的 ISP，特别是类似小型企业的商业客户和家庭客户，对 IP 地址的需求很快超出了可用的地址块。

IP 地址短缺问题并不是一个只有在将来某个时候才可能发生的理论问题。现在，此时此地，这个问题已经发生。对于整个 Internet 而言，长期的解决方案是迁移到 IPv6，它有 128 位地址。这个迁移过程正在缓慢地进行着，可能需要很多年才能完成。为了在这期间解决地址短缺问题，需要一个速战速决的方案。今天普遍使用的快速方案就是网络地址转换（NAT，Network Address Translation）形式，RFC 3022 描述了 NAT，下面我们简要介绍这项技术。有关更多的信息，请参考（Dutcher，2001）。

NAT 的基本思想是 ISP 为每个家庭或每个公司分配一个 IP 地址（或者，最多分配少量的 IP 地址），用这个 IP 地址来传输 Internet 流量。在客户网络内部，每台计算机有唯一的 IP 地址，该地址主要用来路由内部流量。然而，当一个数据包需要离开客户网络，发向其他 ISP 时，它必须执行一个地址转换，把唯一的内部 IP 地址转换成那个共享的公共 IP 地址。这种地址转换使用了 IP 地址的三个范围，这些地址已经被声明为私有化。任何网络可以在内部随意地使用这些地址。仅有的规则是不允许包含这些地址的数据包出现在 Internet 上。这 3 个保留的地址范围为：

10.0.0.0~10.255.255.255/8	(16 777 216 个主机)
172.16.0.0~172.31.255.255/12	(1 048 576 个主机)
192.168.0.0~192.168.255.255/16	(65 536 个主机)

第一段地址提供了 16 777 216 个地址（按常规，去掉全 0 和全 1 的地址），即使网络不那么大，通常大家都喜欢选择这个地址。

NAT 的操作过程如图 5-55 所示。在客户办公室内，每台机器都有一个形如 10.x.y.z 的

地址。然而，当一个数据包离开客户驻地时，它首先要通过一个 **NAT 盒子**（NAT box），此 NAT 盒子将内部的 IP 源地址（图中为 10.0.0.1）转换成该客户所拥有的真实 IP 地址，在本例中为 198.60.42.12。NAT 盒子通常与防火墙组合成一个单一设备，防火墙提供了一种安全机制，它仔细地控制着进出客户网络的流量。我们将在第 8 章学习防火墙知识。另外还有一种可能，将 NAT 盒子集成到路由器或者 ADSL 调制解调器中。

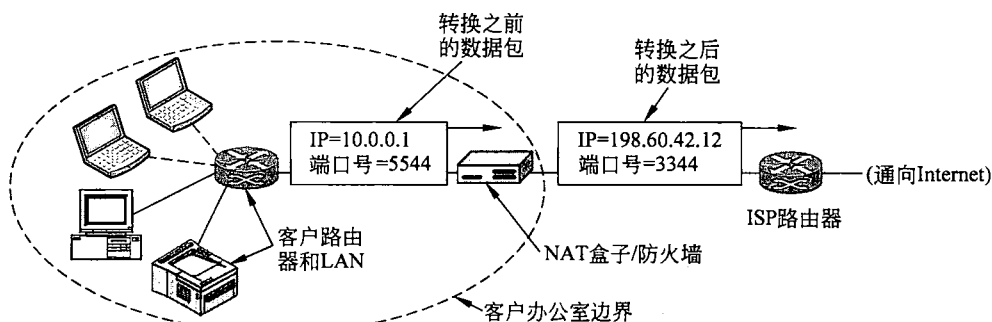


图 5-55 NAT 盒子的放置和操作过程

至此，我们掩盖了一个微小但至关重要的细节：当应答数据包返回时（比如从 Web 服务器返回的应答包），本质上它的目标地址是 198.60.42.12，那么，NAT 盒子如何知道该用哪一个地址来替代呢？这就是 NAT 需要解决的问题。如果在 IP 头有一个备用的字段，那么 NAT 盒子就可以用该字段记录真正的发送方到底是谁，但是在 IP 头仅仅只有 1 位还没有用。从原理上来说，我们还可以创建一个新的 IP 头选项来存放真实的源地址，但是，这么做将要求改变整个 Internet 上所有机器的 IP 代码，使得这些机器都能够处理新的选项。这显然也不是一种有效的快速解决方案。

NAT 的实际做法如下所述。NAT 设计者们注意到，大多数 IP 数据包携带的要么是 TCP 有效载荷，要么是 UDP 有效载荷。当我们在第 6 章学习 TCP 和 UDP 时，将会看到这两个协议的头都包含了一个源端口和一个目标端口字段。下面我们只讨论 TCP 端口，但是所讨论的内容同样适用于 UDP 端口。这里的端口是 16 位整数，它指示了 TCP 连接从哪里开始，以及到哪里结束。正是这些端口字段为 NAT 提供了工作所需的字段。

当一个进程希望与另一个远程进程建立 TCP 连接时，它把自己绑定到一个本地机器尚未使用的 TCP 端口上。该端口称为源端口（source port），它告诉 TCP 代码凡是属于该连接的入境数据包都应该发送给该端口。这个进程还要提供一个目标端口（destination port），以指明数据包传输到远程机器上之后应该交给谁处理。0~1023 之间的端口都是保留端口，用于一些知名的服务。例如，端口 80 是 Web 服务器使用的端口，所以远程客户能找到 Web 服务器进程。每个出境 TCP 消息都包含一个源端口和一个目标端口。这两个端口合起来标识了客户机和服务器两端正在使用该连接的一对进程。

打个比方也许能更清楚地说明端口的用途。假设一家公司只有一个主电话号码。当人们拨打该号码，他们的呼叫到达接线员那里，接线员问他们要哪个分机，然后为他们接通所需的分机。这里的主号码就好比是公司的 IP 地址，两端的分机号就好比是端口。端口实际上是另一个 16 位的地址值，它标识了哪个进程接受入境数据包。

利用 Source port（源端口）字段，我们可以解决前面的映射问题。任何时候当出境数

据包进入 NAT 盒子, 其源地址 10.x.y.z 被客户的真实 IP 地址所取代, 而且, TCP 的 Source port 字段被一个索引值取代, 该索引值指向 NAT 盒子的地址转换表中 65 536 个表项之一。该表项包含了原来的 IP 地址和原来的源端口。最后, NAT 盒子重新计算 IP 头和 TCP 头的校验和, 并将校验和插入到数据包中。这里之所以要替换 source port 域, 是因为来自机器 10.0.0.1 和 10.0.0.2 连接可能碰巧使用了同一个端口, 比如都使用了 5000, 所以仅仅使用 Source port 还不足以唯一性地标识发送进程。

当一个数据包从 ISP 到达 NAT 盒子时, Source port 从 TCP 头中提取出, 并被用作索引值查找 NAT 盒子的映射表。找到对应的表项后, 从该表项提取出内部 IP 地址和原来的 TCP Source port, 并将它们插入到数据包中。然后重新计算 IP 和 TCP 校验和, 并插入到数据包中。最后将该数据包传递给客户内部的路由器, 它使用 10.x.y.z 地址进行正常的路由。

虽然这种方案从某种程度上解决了问题, 但是, IP 团体中的网络纯粹主义者认为它不伦不类 (地球上的丑八怪)。简单地概括, 这里列举出一些反对意见。第一, NAT 违反了 IP 的结构模型。IP 的结构模型声明每个 IP 地址均唯一标识了世界上的一台机器。Internet 的软件结构也是建立在这样的事实基础之上的。采用了 NAT 之后, 成千上万台机器可能会使用地址 10.0.0.1 (事实上也确实如此)。

第二, NAT 打破了 Internet 的端-端的连接模型, 即任何一个主机可在任何时间给任何一台其他主机发送数据包。因为 NAT 盒子上的映射是由出境数据包建立的, 只能在出境数据包之后到达的入境数据包才能被接受。实际上, 这意味着, 家庭网络用户可以通过 NAT 与一台远程 Web 服务器建立 TCP/IP 连接, 但远程用户无法与家庭网络内的一台游戏服务器建立连接, 这种情况需要特殊的配置技术或者 NAT 穿越 (NAT traversal) 技术。

第三, NAT 将 Internet 从一个无连接网络改变成一个面向连接网络特有的形式。问题在于 NAT 盒子必须为每一个从它这里经过的连接维护必要的信息 (即映射关系)。让网络维护连接状态是面向连接网络的一种特性, 而不是无连接网络的特性。如果 NAT 盒子崩溃, 并且它的映射表丢失, 那么它的所有 TCP 连接都将被摧毁。没有 NAT 的情况下, 路由器可以崩溃并重启, 对 TCP 连接没有长远影响。发送进程只是在几秒钟内发生超时, 然后重传所有未被确认的数据包。使用了 NAT 之后, Internet 如同电路交换网络一样, 变得非常脆弱。

第四, NAT 违反了最基本的协议分层规则: 第 k 层不应该对第 $k+1$ 层在本层的有效载荷字段中放什么作任何假设。这条基本原则可以保证层与层之间的独立性。如果 TCP 后来升级到 TCP-2, 它的头结构发生了变换 (比如使用 32 位的端口号), 那么 NAT 将不能再正常工作。分层协议的总体思想是保证某一层的变化不会要求其他层也跟着改变。NAT 破坏了这种独立性。

第五, Internet 上的进程并不一定必须使用 TCP 或者 UDP。如果机器 A 上的一个用户决定使用一种新的传输协议与机器 B 上的用户进行通话 (比如, 一个多媒体应用), 那么, 由于 NAT 的介入, 这样的应用将无法工作, 因为 NAT 盒子将无法正确地定位到 TCP 的 Source port。

第六及相关问题是, 有些应用以规定的方式使用多个 TCP/IP 连接或者 UDP 端口。例如, 标准的文件传输协议 (FTP, File Transfer Protocol) 在数据包正文插入 IP 地址, 接收

方正文中提取出这些地址, 并使用它们。由于 NAT 对这些地址安排一无所知, 它不可能重写这些 IP 地址, 或者说明情况。无法理解就意味着除非采取特殊的防范措施, 否则 FTP 和其他一些应用, 比如 H.323 Internet 电话协议 (我们将在第 7 章讨论该协议), 在必须通过 NAT 的情况下不能正常工作。一般可能的做法是在 NAT 盒子上打补丁。但是, 每次有新应用出现时都要为 NAT 盒打补丁显然不是一个好主意。

最后, 由于 TCP Source port 字段 16 位长, 所以, 至多只有 65 536 台机器可以被映射到同一个 IP 地址上。实际上, 这个数值还要略小一些, 因为前 4096 个端口被保留作特殊的用途。然而, 如果有多个 IP 地址可用, 那么, 每个地址可以处理多达 61 440 台机器。

RFC 2993 针对 NAT 的这些问题和其他一些问题进行了讨论。尽管存在问题, 实际上, 作为处理 IP 地址短缺的权宜之计技术, NAT 已被广泛应用, 特别是家庭和小型企业网络。它已与防火墙和隐私性结合在一起, 因为它能默认阻止未经请求的入境数据包。出于这个原因, 甚至当 IPv6 被广泛部署后, 它也不太可能消失。

5.6.3 IPv6 协议

IP 已被大量使用了几十年。IP 工作得相当好, Internet 的指数级增长就是一个佐证。不幸的是, IP 已经成为其自己知名度的牺牲品: 地址即将殚精竭思。即使通过 CIDR 和 NAT 使用地址更加谨慎保守, 预计最后 IPv4 地址有望在 2012 年底前被 ICANN 分配完毕。这种迫在眉睫的灾难几乎 20 年前就得到了大家的公认, 它在 Internet 社团内部引发了大量应该做些什么的讨论和争议。

在本节, 我们将介绍两个问题, 并提出若干解决方案。唯一的长期解决方案是移动到更大的地址空间。IPv6 (IP 版本 6) 就是能做到这点的一个替换设计。它采用 128 位地址, 在可预见的将来任何时间都不可能出现地址短缺这个问题。但是, IPv6 已经被证明其部署非常困难。这是一个不同的网络层协议, 尽管和 IPv4 有许多相似之处, 但它并没有真正与 IPv4 实现互通。此外, 公司和用户真的不知道为什么他们应该在任何情况下都要用 IPv6。其结果是虽然部署了 IPv6, 但只有 (估计为 1%) 一小部分的 Internet 在使用, 而相应的 Internet 标准 1998 就已经发布。今后几年将会非常有趣, 因为能分配的剩余 IPv4 地址已经为数不多了。人们会在 eBay 上开始拍卖自己的 IPv4 的地址? 会因此而崛起一个 IP 地址黑市吗? 天知道!

除了这些技术问题以外, 还有其他隐藏在背后的问题。早期的 Internet 主要被大学、高科技工业和美国政府 (特别是美国国防部) 使用。20 世纪 90 年代中期开始, 随着对 Internet 兴趣的不断膨胀, Internet 开始为各种人群所使用, 通常人们的需求各不相同。首先, 大量携带智能手机的用户通过 Internet 与他们的家庭基地 (home bases) 保持联系。其次, 随着计算机、通信和娱乐业的不断交融, 有可能在不久的将来, 世界上的每一部电话和每一个电视都变成了 Internet 节点, 从而几十亿台机器可进行音频和视频点播。很显然, 在这样的形势下, IP 必须要演进, 要变得更加灵活。

1990 年这些问题初露端倪, IETF 开始启动 IP 新版本的设计工作, 新版本的 IP 将有不完的地址, 并要解决许多其他的问题, 同时必须更加灵活和高效。它的主要目标是:

- (1) 即使地址空间的分配效率不高, 也能支持几十亿台主机。

- (2) 降低路由表的大小。
- (3) 简化协议, 使路由器更快速地处理数据包。
- (4) 提供更好的安全(认证和隐私)。
- (5) 更加关注服务类型, 特别是针对实时数据。
- (6) 辅助指定范围内的组播。
- (7) 主机漫游时无须改变地址。
- (8) 允许协议向未来演进。
- (9) 允许新老协议共存多年。

IPv6 协议的设计提出了一个重大机遇, 可以借此机会改善 IPv4 缺少但现在又需要的所有特性。为了开发出一个满足所有这些需求的协议, IETF 在 RFC 1550 中发出了一个寻求提案和讨论的呼吁, 刚开始共收到 21 份响应材料。到 1992 年 12 月, 有 7 个提案被拿到了桌面上讨论。这些提案相差甚大, 涉及范围非常广泛, 从“对 IP 做微小的修改”, 到“完全抛掉 IP 而用一个全然不同的协议来替代”。

其中一种提案是在 CLNP 之上运行 TCP。CLNP 是一个由 OSI 设计的网络层协议, 它有 160 位地址, 因为它可为海洋中的每个分子都分配足够的地址用来建立小型网络(大约为 2^5), 所以地址空间永远够用。而且这种选择还将统一两个主要的网络层协议。然而, 许多人认为, 这样做好像承认了 OSI 领域所做的事情实际上是正确的, 而在 Internet 圈子中却存在策略性的错误。CLNP 的模式与 IP 非常相近, 所以这两种协议并没有实质性的不同。实际上, 最终选中的协议与 IP 之间的差异, 远远超过了 CLNP 与 IP 之间的差异。反对 CLNP 的另一个理由是它对服务类型的支持太差, 而这对于有效传输多媒体数据是非常必要的。

IEEE Network 发表了三种比较好的提案(Deering, 1993; Francis, 1993; Katz 和 Ford, 1993)。在经过多次讨论、修订和定位之后, Deering 和 Francis 两份提案被组合起来并又作了修改, 然后得到一个现在称为简单 Internet 协议+ (SIPP, Simple Internet Protocol Plus) 的协议, 最终, 它被选中, 并指定为 IPv6。

IPv6 很好地满足了 IETF 的设计目标。它保持了 IP 的优良特性, 丢弃或者削弱了 IP 中不好的特性, 并且在必要的地方增加了新的特性。一般而言, IPv6 并不与 IPv4 兼容; 但是它与其他一些辅助性的 Internet 协议则是兼容的, 包括 TCP、UDP、ICMP、IGMP、OSPF、BGP 和 DNS; 若要处理更长的地址则需要做一点小小的改动。下面讨论 IPv6 的主要特性, 有关更多的信息可以在 RFC 2460~2466 中找到。

第一个, 也是最重要的是 IPv6 有比 IPv4 更长的地址。IPv6 的地址 128 位长, 这解决了 IPv6 一开始就想要解决的问题: 提供一个有效的无限量 Internet 地址。稍后我们马上还要更多地谈论地址。

IPv6 的第二个主要改进是对头进行了简化。它只包含 7 个字段(相比之下, IPv4 有 13 个字段)。这一变化使得路由器可以更快地处理数据包, 从而提高了吞吐量, 并缩短延迟。同样地, 我们后面还要讨论 IPv6 的头结构。

第三个主要改进是更好地支持选项。这一变化对于新的头来说是本质的, 因为以前那些必需的字段现在变成了可选; 而且选项的表达方式也有所不同, 这使得路由器可以非常简单地跳过那些与它无关的选项。此特性也加快了数据包的处理速度。

第四个改进代表了 IPv6 的重大进步, 即在安全性方面的改进。IETF 已经听腻了关于

早熟的 12 岁少年用他们的个人计算机闯入 Internet 银行和军事堡垒的新闻故事，因此它们存在着一种强烈的意识，要在增强安全性方面做点事情。在新的 IP 中，认证和隐私是安全方面的关键特征。然而，后来这些特征也被引入到 IPv4 中，所以 IPv6 和 IPv4 在安全性方面的差异已经没有那么大了。

最后，更加值得关注的是服务质量。过去，人们在这方面做了大量半心半意的努力来改善 QoS，但是现在，随着多媒体在 Internet 上的增长，这种紧迫性更加强烈了。

主要的 IPv6 头

IPv6 的头如图 5-56 所示。对于 IPv6，版本（Version）字段总是 6（对于 IPv4，该字段总是 4）。在从 IPv4 到 IPv6 的迁移过程中（这个过程已经持续了 10 年），路由器通过检查该字段来确定数据包的类型。顺便提一下，做这样的测试在关键路径上需要浪费少量的指令，所以，一些路由器可能会跳过该项检查，数据链路层的头通常指名了多路分用中的网络层协议。例如，以太网的 Type 字段针对 IPv4 有效载荷和 IPv6 有效载荷，分别给出了不同的值。在“做得对”（Do it right）与“做得快”（Make it fast）两大阵营之间的讨论无疑将会既漫长又激烈。

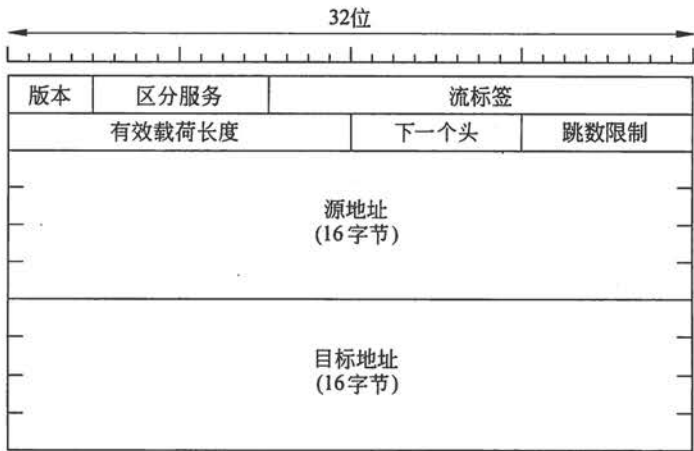


图 5-56 IPv6 固定头（必需的）

区分服务（Differentiated services，最初称为流量类别）字段的用途主要是区分数据包的服务类别，这些数据包具有不同的实时传递需求。它主要被用在服务质量的区分服务体系中，使用方式与 IPv4 数据包的同名字段一样。此外，最低 2 位用来发送显式拥塞指示，也与 IPv4 的方式相同。

流标签（Flow label）字段为源端和接收方提供了一种建立伪连接的方式，即源端和接收方把一组具有同样需求并希望得到网络同等对待的数据包打上标记。例如，从某台特定主机上一个进程到一台特定主机上一个进程之间的数据包流可能有严格的延迟要求，因此需要预留带宽。这时可以提前设置一个流（flow），并分配一个标识符。当一个流标签字段非 0 的数据包出现时，所有的路由器都在自己的内部表中查找该流标签值，看它要求哪一种特殊的待遇。实际上，这样的流是两种传输模型相结合的一种尝试：数据报网络的灵活性和虚电路网络的保障性。

为了保障服务质量，每个流由源地址、目标地址和流编号来指定。这意味着在给定的 一对 IP 地址之间，可以同时有 220 个活跃的流。而且还意味着来自不同主机的两个流即使有相同的流标签，当它们通过同一台路由器时，路由器也能够根据源地址和目标地址将它们区分开。流标签的选取最好随机，而不是从 1 开始顺序分配，因此，路由器可对它们进行哈希处理。

有效载荷长度 (Payload length) 字段指明了紧跟在图 5-56 中 40 个字节头之后还有多少字节数。在 IPv4 中该字段的名称为总长度 (Total length)，之所以改成现在的名称是因为含义略有不同：40 字节的头不再像以前那样算作长度中的一部分。

正是下一个头 (Next header) 字段显示了 IPv6 的与众不同关键之处。IPv6 头得以简化的原因在于它可以有额外的 (可选) 扩展头。该字段指明了当前头之后还有哪种扩展头 (当前已经定义了 6 种扩展头)，如果有的话。如果当前的头是最后一个 IP 头，那么下一个头字段指定了该数据包将被传递给哪个传输协议处理 (比如 TCP、UDP)。

跳数限制 (Hop limit) 字段被用来避免出现数据包永垂不朽的情形。实际上，它与 IPv4 中的 TTL (Time to live) 字段是一样的，也就是说，在每一跳上该字段中的值都要被路由器递减。理论上，IPv4 中的 TTL 是一个以秒为单位的时间值，但是所有的路由器都不按照时间值来操作，所以在 IPv6 中将名字作了修改，以便反映出它的实际用法。

接下来是源地址 (Source address) 和目标地址 (Destination address) 字段。在 Deering 的原始提案 (SIP) 中，他提出使用 8 字节地址，但是在提案审阅过程中，许多人认为，如果用 8 个字节作为 IPv6 中的地址，那么在几十年之内地址空间将再次被用完，而如果使用 16 字节的地址，则永远也用不完；其他人则认为 16 个字节是矫枉过正，而还有些人主张使用 20 字节的地址以便与 OSI 数据报协议兼容；甚至还有人建议使用可变长度的地址。经过了太多的讨论和无法在教科书上说明的争执之后，最终做出决定，固定长度的 16 字节地址是最好的折中方案。

为了便于书写 16 字节的地址，一种新的标记法被设计了出来。16 个字节被分成 8 组来书写，每一组 4 个十六进制数字，组之间用冒号隔开，如下所示：

8000:0000:0000:0000:0123:4567:89AB:CDEF

由于许多地址的内部可能有很多个 0，所以，三种优化方法获得授权。第一，在一个组内可以省略前导 0，因此 0123 可以写成 123；第二，16 个“0”构成的一个或多个组可以用一对冒号来代替，因此，上面的地址现在可以写成：

8000::123:4567:89AB:CDEF

第三，IPv4 地址现在可以写成一对冒号再加上老式的点分十进制数，例如：

::192.31.20.46

弄清楚 IPv6 有多少个地址可能并没有必要，但是 16 字节的地址确实非常多。具体来说，应该有 2^{128} 个地址，近似等于 3×10^{38} 个地址。如果整个地球，包括陆地和水面都被计算机覆盖，那么，IPv6 将保证每平方米有 7×10^{25} 个地址。化学系的学生将会注意到，这个数值超出了阿伏伽德罗常数 (Avogadro's number)。虽然 IPv6 的目标并不是要为地球表面上的每一个分子都分配单独的 IP 地址，但我们离那个目标并不遥远。

实际上，地址空间的使用效率不会非常高，就好像电话号码空间一样 (曼哈顿的区号 212 几乎已经用满，而怀俄明州的区号 307 几乎还是空的)。在 RFC 3194 中，Durand 和

Huitema 对此作了计算，他们利用电话号码分配方案作为参照，经计算得到的结果是即使在最不利的情形下，地球表面（包括陆地和水面）上每平方米仍将有远远超过 1000 个 IP 地址。在任何可能的场景下，每平方米将有几万亿个 IP 地址。总而言之，在可预见的将来，我们不太可能用得完这些地址。

比较 IPv4（图 5-46）与 IPv6 头（图 5-56）看看在 IPv6 中省掉了什么非常有意义。IHL 字段没有了，因为 IPv6 头有固定的长度。协议（Protocol）字段也被拿掉了，因为下一个头（Next header）字段指明了最后的 IP 头后面跟的是什么（比如 UDP 或者 TCP 段）。

所有与分段有关的字段都被去掉了，因为 IPv6 采用了另一种方法来实现分段。首先，所有遵从 IPv6 的主机都应该能够动态地确定将要使用的数据包长度。主机使用我们在 5.5.5 节描述的路径 MTU 发现过程就能做到这点。简要地说，当主机发送了一个非常大的 IPv6 数据包时，如果路由器不能转发这么大的数据包，它并不对该数据包进行分段，而是向发送主机返回一条报错消息。这条消息告诉主机，所有将来发送给该目标地址的数据包都要分段。让主机从一开始就发送大小合适的数据包，比让沿途路由器动态地对每个数据包进行分段要有效得多。而且，最小数据包长度也从 576 字节增加到 1280 字节，以便允许 1024 字节的数据和许多个头。

最后，校验和（Checksum）字段也被去掉了，因为计算校验和会极大地降低性能。现在使用的是大多是可靠网络，而且数据链路层和传输层通常有它们自己的校验和，所以在网络层上再使用校验和，相比它所付出的性能代价是不值得的。去掉了所有这些特性之后得到的是一个精简的网络层协议。因此，这份设计方案满足了 IPv6 的目标，即一个快速，但灵活，并且具有足够大地址空间的协议。

扩展头

有些省略掉的 IPv4 字段偶尔还会有用，所以，IPv6 引入了（可选的）扩展头（extension header）这一概念。这些扩展头可以用来提供一些额外的信息，但是它们以一种更有效的方式编码。目前定义了 6 种扩展头，如图 5-57 所列。每一种扩展头都是可选的，但是如果多个扩展头出现，那么它们必须直接跟在固定头部的后面，而且最好使用表中列出的顺序。

扩展头	描述
逐跳选项	路由器的混杂信息
目标选项	给目的地的额外信息
路由	必须访问的松散路由器列表
分段	管理数据报分段
认证	验证发送方的身份
加密安全有效载荷	有关加密内容的信息

图 5-57 IPv6 扩展头

有些扩展头有固定的格式，其他扩展头包含数目不定的可变长度选项。对于所有这些可变选项，每一项都被编码成一个（Type, Length, Value）三元组。Type（类型）字段占一个字节，它指明这是什么选项。Type 的值有特殊的选取方法，它的前 2 位告诉路由器如果不知道如何处理该选项时应该如何处置数据包。选择方案有：跳过此选项；丢弃该数据包；丢弃该数据包并返回一个 ICMP 数据包；丢弃该数据包，但是对于组播地址不发送 ICMP

数据包（这样可以避免一个坏的组播数据包产生数百万个 ICMP 报告。）

长度（Length）字段也占一个字节，它说明了 Value 字段有多长（从 0~255 个字节）。值（Value）字段是任何扩展头所需要的信息，可以长达 255 个字节。

逐跳头（hop-by-hop header）用来存放沿途所有路由器必须要检查的信息。到现在为止，已经定义了一个选项：支持超过 64 KB 的数据报。该头的格式如图 5-58 所示。使用这种扩展头时，固定头中的有效载荷长度（Payload length）字段要设置为 0。

下一个头	0	194	4
巨型有效载荷长度			

图 5-58 用于大型数据报（巨型报）的逐跳扩展头

与所有的扩展头一样，逐跳扩展头的起始字节也指定了接下去是哪一种头。该字节之后的字节指示了当前逐跳扩展头有多少个字节，其中不包括起始的 8 个字节，因为这 8 个字节是强制的。所有的扩展头都是以这种方式开始。

接下去的两个字节表明该选项定义了数据报的长度（代码 194），并且长度值以 4 字节计数。最后 4 个字节给出了数据报的长度。小于 65 535 的长度值是不允许的，第一台路由器将会丢弃这样的数据包，并且返回一个 ICMP 错误消息。使用这种扩展头的数据报称为巨型数据报（jumbogram），对于那些必须要通过 Internet 传输千兆字节数据的超级计算机应用来说，巨型数据报的使用非常重要。

目标选项扩展头（destination options header）用于那些只需被目标主机翻译的字段。在 IPv6 的初始版本中，唯一定义的选项是空选项（null option）。利用空选项可以将当前头拉长到 8 字节的倍数，所以它最初没有被使用。之所以设置这个选项是为了以防万一，也许某一天有人会想到一种新的目标选项，这样就能确保新的路由软件和主机软件可以对它进行处理。

路由扩展头列出了在通向目标的途中必须要经过的一台或者多台路由器。它非常类似于 IPv4 的松散源路由。在松散源路由机制中，凡是列出来的地址，必须要严格按顺序被访问到，但是，这些地址中间也可以经过一些没有列出来的其他路由器。路由头的格式如图 5-59 所示。

下一个头	扩展头长度	路由类型	剩余段数
特定类型的数据			

图 5-59 路由扩展头

路由扩展头的前 4 个字节包含了 4 个单字节整数。下一个头（Next header）和扩展头长度（Header extension length）字段如上面所述。路由类型（Routing type）字段给出了该扩展头剩余部分的格式。类型 0 表示在第一个字后面是一个保留的 32 位字，然后是一定数量的 IPv6 地址。将来根据需要还可以发明其他的类型。最后，剩余段数（Segments left）字段记录了在地址列表中还有多少个地址尚未被访问到。每当一个地址被访问时，该字段

中的数值减一。当它被减到 0 的时候，该数据包就完全获得自由，它不需要再遵循任何路由路径了。通常到这个时候它离目标已经非常接近，所以最佳路径也非常显然了。

分段扩展头（Fragment header）涉及与分段有关的事项，其处理方法与 IPv4 的做法非常类似。该扩展头保存了数据报的标识符、分段号，以及指明了后面是否还有更多段的标志位。然而，与 IPv4 不同的是，在 IPv6 中，只有源主机才可以将一个数据包进行分段。沿途的路由器可能不会进行分段。这一改变是对原始 IP 的重大哲学突破，但是符合 IPv4 的现行做法；而且它简化了路由器的工作，使得路由过程更快。正如上面所提到的，如果路由器面临一个太大的数据包，那么它可以丢弃该数据包并且向源主机发回一个 ICMP 包。这一信息允许源主机使用本扩展头把数据包分割成小的片段，然后再试着重新发送。

认证扩展头（Authentication header）提供了一种让数据包接收方确定发送方身份的机制。加密安全有效载荷扩展头（Encrypted security payload header）使得有可能对数据包的内容进行加密，因此，只有真正的接收方才可以读取数据包内容。这两个扩展头使用密码学技术来完成它们的任务。

争论

鉴于开放式设计过程和所涉及许多人的强烈意见，毫不奇怪，IPv6 做了很多极具争议性的选择。下面我们简短地对这些争议做一番总结。要想了解全面的细节，请参考有关的 RFC 文档。

我们前面已经提到了有关地址长度的争论，最终的结果是一个折中的方案：16 字节固定长度的地址。

另一个争论发生在跳数限制（Hop limit）字段。有一方强烈地认为，将最大跳数限制在 255 以内（也就是说，用一个 8 位的字段）是一个很显然的错误。毕竟，32 跳的路径现在非常普遍，10 年以后比这更长的路径也会变得十分普遍。这些人争辩说，使用大的地址长度很有远见，但是，使用小的跳计数则非常短视。在他们眼里，计算机科学家能够犯的最大错误就是在有些地方提供太少的数据位。

他们得到的回应是一旦贯彻就可能要增加每一个字段，从而导致一个非常臃肿的头部。跳数限制字段用来避免数据包长时间地逗留在网络中，但 65 535 跳实在是太长了。而且，随着 Internet 的增长，越来越多的长距离链路将被建立起来，从而使得从一个国家到达另一个国家可能至多不超过六跳。如果从源端和接收方分别到达它们相应的国际网关需要超过 125 跳，那么它们的国家主干网一定有问题了。所以，最终 8 位支持者们赢得了胜利。

另一个棘手的问题是最大数据包长度。超级计算机社团希望数据包长度可以超过 64 KB。当一台超级计算机开始传输数据时，它的业务就开始了，当然不希望每 64 KB 被中断一次。反对派的观点是如果一个 1MB 的数据包被送到一条 1.5 Mbps 的 T1 线路上，那么该数据包将会占用线路 5 秒钟，对于共享同一线路的交互用户而言，这是一个明显感觉得到的延迟。在这一点上，最终达成的一致意见是：正常的数据包被限制在 64 KB 以内，但是允许使用逐跳扩展头来传送巨型数据报。

第三个热点话题是去掉 IPv4 校验和。有些人把这种做法比作从一辆汽车上拆除了刹车。拆除了刹车以后，汽车轻多了，所以它可以跑得更快；但如果发生意外事件，那就有问题了。

校验和反对派的论点是任何真正关心数据完整性的应用，不管用什么方法，一定要有一个传输层校验和，所以在 IP 中使用另一个校验和（而且还有一个数据链路层的校验和）纯属多余。更进一步，经验表明计算 IP 校验和是 IPv4 协议的一个主要开销。反对校验和阵营最终赢得了胜利，所以 IPv6 没有保留校验和。

移动主机也是一个争论的焦点。如果一台便携式计算机正在世界各地旅行，那么，它是继续使用同样的 IPv6 地址还是必须使用需要家乡代理支持的方案呢？有人希望在 IPv6 中对移动主机提供显式支持。由于任何一个提案都没有达成一致的意见，所以最终这一努力以失败而告终。

可能最大的争论在于安全性。每个人都承认安全性非常重要，争议的焦点在于在哪里和如何实现安全。首先是在哪里实现安全性。支持在网络层实现安全性的观点是，安全性变成一种标准的服务，于是所有的应用都可以使用安全服务，而无须任何提前规划。反对派则认为，真正的安全应用一般只需要端到端的加密，其中源端应用完成加密过程，接收方应用完成解密过程。网络层的实现可能会有错误，用户对此没有任何控制能力却要受到它的约束。对这种观点的回应是这些应用可以不使用 IP 的安全特性，改而自己实现端到端的安全性。对方又反驳道，那些不信任网络的人自己实现了安全性，即使禁用了网络层的安全功能也不想为这一慢速而又庞大的 IP 实现付出额外的费用。

与何处实现安全性相关的另一个话题是许多（并非全部）国家都有针对密码系统的严格出口法规。有些国家，特别是法国和伊拉克，限制在国内使用密码系统，因此老百姓对政府来说没有任何秘密可守。结果，任何一个使用了一定强度密码系统的 IP 实现不可能从美国（和许多其他的国家）出口给全球的客户。为此，软件厂商不得不维护两套软件，一套给国内客户使用，另一套出口到国外，这遭到了许多计算机厂商的强烈反对。

有一点倒是大家都没有任何异议，那就是没有人期望在某个星期天晚上 IPv4 Internet 被突然关闭，然后星期一早晨切换到 IPv6 Internet。相反，对被隔离的 IPv6 “孤岛”进行改造，刚开始的时候可以通过隧道方式进行通信，我们在 5.5.3 节描述了隧道技术。随着 IPv6 岛的增加，它们将合并成更大的岛，最终所有的岛屿都合并到一起，于是 Internet 将完全转变为 IPv6 过来。

至少，这是个计划。已有的部署证明了 IPv6 的死穴。即使所有主要操作系统完全支持它，它也仍然很少被使用。大多数的 IPv6 部署是为了满足网络运营商需要大量 IP 地址的新情形，例如移动电话运营商。已经定义了许多策略，以便有助于缓解过渡压力。其中一些方法可以被主机用来自动配置隧道，通过 IPv4 Internet 运载 IPv6；还有一些方法可使主机自动找到隧道出入口。双栈主机同时实现了 IPv4 和 IPv6 协议，可以根据数据包的目的地址选择要使用的协议版本。这些战略将大幅简化 IPv6 的后续部署，当 IPv4 地址耗尽时 IPv6 的大量部署似乎是不可避免的。有关 IPv6 的更多信息，请参阅（Davies, 2008）。

5.6.4 Internet 控制协议

除了用于数据传输的 IP 协议外，Internet 在网络层还有几个辅助控制协议。它们包括 ICMP 协议、ARP 协议和 DHCP 协议。在本节，我们将依次考查每个协议，给出对应于 IPv4 的协议描述，因为这些协议是最常用的。针对 IPv6，ICMP 和 DHCP 有类似的版本，而与

ARP 等价的协议则称为 NDP（邻居发现协议）。

ICMP——Internet 控制消息协议

路由器严密监视 Internet 的操作。当路由器在处理一个数据包的过程中发生了意外，可通过 Internet 控制消息协议（ICMP，Internet Control Message Protocol）向数据包的源端报告有关事件；ICMP 还可以用来测试 Internet。已经定义的 ICMP 消息类型大约有 10 多种，每一种 ICMP 消息类型都被封装在一个 IP 数据包中。图 5-60 列出了最重要的一些消息类型。

消息类型	描述
目的地不可达	数据包无法传递
超时	TTL 字段减为 0
参数问题	无效的头字段
源抑制	抑制包
重定向	告知路由器有关地理信息
回显和回显应答	检查一台机器是否活着
请求/应答时间戳	与回显一样，但还要求时间戳
路由器通告/恳求	发现附近的路由器

图 5-60 主要的 ICMP 消息类型

当路由器不能定位一个目标，或者当一个设置了 DF 标志位的数据包由于途中经过一个“小数据包”网络而不能被递交时，路由器可以使用目的地不可达（DESTINATION UNREACHABLE）消息来报告这种情况。

当一个数据包由于它的 TTL（生存期）达到 0 而被丢弃时，路由器发送超时（TIME EXCEEDED）消息。这种事件往往预示着数据包进入了路由循环，或者计时器的超时值设置得太小。

这个错误信息可巧妙地被用于 Traceroute 工具，该工具由 Van Jacobson 在 1987 年开发。Traceroute 可发现从主机到目的地的路径上的 IP 地址，而且不需要任何特权网络的支持就能发现这种信息。方法其实很简单：给目标地址发送一系列的数据包，分别将 TTL 设置为 1、2、3，以此类推。这些数据包的计数值沿路径延伸而被后续路由器逐步递减为零，因此这些路由器各自乖乖地发送一个超时消息给发送主机。根据收到的这些返回信息，主机就可以确定路径沿途的路由器 IP 地址，以及跟踪路径各部分的统计数据和时间开销。这不是超时消息的本意，但它也许是最有用的网络调试工具。

参数问题（PARAMETER PROBLEM）消息表示在头字段中检测到一个非法值。这个问题说明了发送主机的 IP 软件中存在错误，或者也可能是中途路由器软件存在错误。

源抑制（SOURCE QUENCH）消息以前被用来抑制那些发送太多数据包的主机。当一台主机接收到这条消息时，它应该将发送速度减慢下来。这种消息现在很少使用了，因为当拥塞发生的时候，再发这些包无疑是火上浇油。现在，Internet 的拥塞控制任务主要由传输层完成；我们将在第 6 章中学习有关的细节。

当路由器注意到一个数据包看起来被错误地路由时，它使用重定向（REDIRECT）消息将可能的错误信息告诉源端主机。

回显 (ECHO) 和回显应答 (ECHO REPLY) 消息可以用来判断一个指定的目标是否可达, 以及是否还活着。目标主机接收到回显消息之后, 应该立即送回一个回显应答消息。这些消息主要被 ping 工具用在探测 Internet 上是否存在某一特定的主机。

请求时间戳 (TIMESTAMP REQUEST) 和应答时间戳 (TIMESTAMP REPLY) 消息的用途类似, 只不过在应答消息中还包含了请求消息的到达时间和应答消息的发出时间。此项设施可以用来测量网络的性能。

路由器通告 (ROUTER ADVERTISEMENT) 和路由器恳求 (ROUTER SOLICITATION) 消息使得主机拥有寻找附近路由器的能力。主机至少需要学习一个路由器的 IP 地址才能发送离开本地网络的数据包。

除了这些消息之外, 标准还定义了其他消息类型。详细的消息类型可在线查询 www.iana.org/assignments/icmp-parameters。

ARP——地址解析协议

尽管 Internet 上的每台机器都有一个 (或多个) IP 地址, 但是仅有这些地址还不足以支撑发送数据包。数据链路层硬件网卡, 比如以太网卡并不理解 Internet 地址。在以太网的环境里, 每一块 NIC 在出厂时都配置了一个唯一的 48 位以太网地址。以太网 NIC 的制造商从 IEEE 请求一块以太网地址, 确保不会出现任何两块网卡有相同的地址 (以避免两块网卡出现在同一个 LAN 上时发生冲突)。NIC 根据其 48 位以太网地址来发送和接收帧。它们对 32 位的 IP 地址完全一无所知。

现在问题来了: 如何将 IP 地址映射到数据链路层的地址, 比如以太网地址呢? 为了解释这一工作过程, 我们来看图 5-61 给出的例子, 这个例子演示了一个规模较小的大学, 它只有两个 /24 网络。计算机系有一个交换式以太网 (CS), 它的前缀是 192.32.65.0/24; 另一个在电子工程学系, 它的前缀为 192.32.63.0/24。这两个局域网通过一个 IP 路由器相连。以太网上的每台机器和路由器上的每个接口都有一个唯一的以太网地址, 我们将它们标记为 CS 或 EE 网络上的 E1 至 E6。

我们来看 CS 网络上主机 1 的用户如何给主机 2 的用户发送数据包。假设发送方知道目标接收方的名字, 可能像 eagle.cs.uni.edu 这样的名字。第一步是找到主机 2 的 IP 地址。这个查找过程由域名系统 (DNS, Domain Name System) 完成, 我们将在第 7 章学习域名系统。此刻, 我们假设 DNS 返回主机 2 的 IP 地址 (192.32.65.5)。

主机 1 上层软件现在构建一个数据包, 其 Destination address 字段为 192.32.65.5, 然后它将该数据包交给 IP 软件来发送。IP 软件看到该地址后发现这个目标地址就在 CS 网络上 (即它自己所在的网络)。然而, 它仍然需要某一种办法来找出目标主机的以太网地址才能发送帧。一种解决方案是在系统中设置一个配置文件, 该配置文件给出了从 IP 地址到以太网地址的映射关系。虽然这种方案当然是可能的, 但是对于拥有几千台机器的组织来说, 在所有主机上保持配置文件并及时更新, 则是一件既容易出错, 又费时的任务。

一个更好的解决方案是主机 1 发送一个广播包到以太网网络上请求拥有 IP 地址 192.32.65.5 的主机。该广播包将会到达 CS 网络上的每一台主机, 并且每台主机都会检查自己的 IP 地址。只有主机 2 会用自己的以太网地址 (E2) 作为应答。通过这种方式, 主机 1 得知 IP 地址 192.32.65.5 是一台拥有以太网地址为 E2 的主机。请求和获得应答两个过程

所使用的协议称为地址解析协议（ARP，Address Resolution Protocol）。几乎 Internet 上的每一台机器都运行这个协议。RFC 826 定义了 ARP。

与采用配置文件相比，使用 ARP 协议的优点是简单。系统管理员只要给每台机器分配一个 IP 地址，并且确定好子网掩码，不用做其他任何事情。ARP 会负责处理好所有其他的事情。

这时候，主机 1 上的 IP 软件构建一个以太网帧，其目标地址为 E2，并且把 IP 数据包（目标地址为 192.32.65.5）放到以太网帧的有效载荷字段中，然后将它发送到以太网上。图 5-61 给出了该数据包的 IP 地址和以太网地址。主机 2 的以太网 NIC 检测到这一帧，并识别出这是发给自己的帧，于是将它接收进来，并产生一个中断；以太网驱动程序从有效载荷中提取出 IP 数据包，并将它传递给 IP 软件，IP 软件看到它的目标地址正是指向自己，于是对它进行处理。

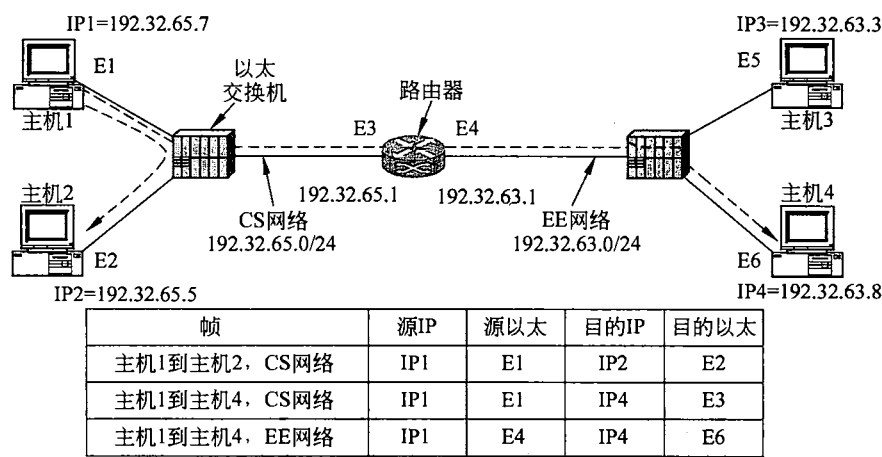


图 5-61 通过路由器联结两个交换式以太 LAN

为了使 ARP 的效率更高，可以进行各种优化处理。首先，一旦一台机器已经运行了 ARP，那么，它可以将结果缓存起来，以便稍后它与同一台机器通信时再用。当下次通信时，它就可以在缓存中找到所需的地址映射关系，从而避免了进行第二次广播。在许多情况下，主机 2 必须返回一个应答，这就迫使它也要运行 ARP 来确定发送方的以太网地址。此 ARP 广播也是可以避免的，因为主机 1 可以将它的“IP-以太网”地址映射关系包含在它的 ARP 包中。当 ARP 广播包到达主机 2 时，(192.31.65.7, E1) 这对映射关系也被存入主机 2 的 ARP 缓存。事实上，以太网上的所有机器都可以将这对映射关系放到它们的 ARP 缓存中。

为了允许映射关系发生变化，比如为一台主机配置了一个新 IP 地址（但保留其老的以太网地址），ARP 缓存中的相应表项应该在几分钟之后发生超时。保持缓存信息最新状态并能优化性能的一种更聪明方式是，让每台机器配置之后广播它的地址映射关系。这次广播通常以 ARP 的形式发送，即主机发送一个 ARP 请求查找它自己的 IP 地址。按理来说，网络上应该不会有任何应答，这个广播数据包的副作用就是其他主机在 ARP 缓存中加入一个映射表项。这就是所谓的免费 ARP（gratuitous ARP）。如果意外地收到了一个应答，那么一定是两台机器被分配了相同的 IP 地址。网络管理员必须解决这个问题，否则这两台机

器都不能使用网络。

现在我们再来看图 5-61, 只不过这一次假设主机 1 想要给 EE 网络上的主机 4 (192.32.63.8) 发送数据包。主机 1 发现目标 IP 地址不在 CS 网络。同时它知道应该把所有这些网络外的流量发给路由器, 该路由器称为默认网关 (Default gateway)。按照惯例, 默认网关具有网络上的最低地址 (198.31.65.1)。为了给路由器发送帧, 主机 1 必须知道该路由器在 CS 网络上的接口地址。因此, 主机 1 发送一个 ARP 广播报文, 请求 198.31.65.1 对应的以太网地址, 从该广播报文的应答报文它获知所需的以太网地址为 E3; 然后用该地址给路由器发送帧。数据包沿着一条 Internet 路径从一个路由器被转发到下一个路由器时就是采用这种查询机制的。

当路由器的以太网 NIC 得到该帧后, 它将数据包交给 IP 软件。IP 软件从网络掩码了解到这个数据包要发送到 EE 网络, 在 EE 网络再到达主机 4。如果路由器不知道主机 4 的以太网地址, 它可以再次使用 ARP。在图 5-61 的表中列出了从 CS 和 EE 网络观察到的帧的源和目标的以太网地址和 IP 地址。我们观察到出现在每个网络上的帧的以太网地址发生了改变, 而 IP 地址保持不变 (因为它们表示所有互联网络的端点)。

还有一种可能, 当主机 1 不知道主机 4 在另一个不同网络时, 仍然可以从主机 1 发送一个数据包给主机 4。解决办法是让 CS 网络上的路由器回答针对主机 4 的 ARP 请求, 并且以 E3 作为响应。直接由主机 4 来响应对自己的 ARP 请求报文是不可能的, 因为它根本看不到 ARP 请求 (路由器不会转发以太网级的广播报文)。然后, 路由器将收到发给 192.32.63.8 的帧, 并将该帧转发到 EE 网络。这个解决方案称为 **ARP 代理** (proxy ARP)。这种方案常用在这样一种特殊情况下: 一个主机想出现在一个网络上, 即使它实际上在另一个网络上。例如, 一个常见的情况是有个移动笔记本, 当它离开家乡网络时, 其他节点仍然能给它发送数据包。

DHCP——动态主机配置协议

ARP (以及其他 Internet 协议) 都做了这样的假设, 即主机配置了一些基本信息, 比如自己的 IP 地址。主机如何获得此信息? 手动配置每台计算机是可能的, 但那既乏味又容易出错。有一个更好的方法可以完成这件事情, 就是**动态主机配置协议** (DHCP, Dynamic Host Configuration Protocol)。

采用 DHCP 时, 每个网络必须有一个 DHCP 服务器负责地址配置。当计算机启动时, 它有一个嵌入在 NIC 中的内置以太网地址或其他链路层地址, 但没有 IP 地址。像 ARP 一样, 该计算机在自己的网络上广播一个报文, 请求 IP 地址。这个请求报文就是 DHCP DISCOVER 包, 这个包必须到达 DHCP 服务器。如果 DHCP 服务器没有直接连在本地网络, 那么必须将路由器配置成能接收 DHCP 广播并将该请求报文中继给 DHCP 服务器, 由 DHCP 服务器来处理 DHCP 报文。

当 DHCP 服务器收到请求, 它就为该主机分配一个空闲的 IP 地址, 并通过 DHCP OFFER 包返回给主机 (这个报文或许也要通过路由器中继)。为了在主机没有 IP 地址的情况下完成此项工作, 服务器用主机的以太网地址来标识这台主机 (主机的以太网地址由 DHCP DISCOVER 包携带过来)。

IP 地址的自动分配有个问题, 就是从地址池中取出一个 IP 地址分配给主机能用多久。

如果一个主机离开网络，并且没有把分配给它的 IP 地址返回给 DHCP 服务器，那么该地址将永久丢失。过一段时间，很多地址都可能因此而丢失。为了防止这种情况发生，可以为每个分配的 IP 地址指定一段固定时间，这种技术称为租赁（leasing）。在租赁期满前，主机必须请求 DHCP 续订。如果没有提出续订请求或请求被拒绝，主机或许不能使用以前分配给它的 IP 地址。

DHCP 由 RFC2131 和 RFC2132 描述。它已被广泛应用于 Internet，可为主机配置除了 IP 地址以外的其他各种参数。它还被用于企业和家庭网络，ISP 使用 DHCP 来设置 Internet 接入链路上的设备参数，因此客户不需要给他们的 ISP 打电话来获得这些信息。常见的配置信息例子包括网络掩码、默认网关的 IP 地址、DNS 服务器和时间服务器的 IP 地址。DHCP 已在很大程度上取代了先前使用的协议（称为 RARP 和 BOOTP），因为这些协议的功能比较有限。

5.6.5 标签交换和 MPLS

到目前为止，在我们的 Internet 网络层巡视中，注意力都专注于数据包，因为这是 IP 路由器转发的数据报。还有另一种技术正在开始广为应用，特别是被 ISP 用来在它们的网络之间移动 Internet 流量。这种技术称为多协议标签交换（MPLS，MultiProtocol Label Switching），它非常接近电路交换。尽管 Internet 社团中的很多人对面向连接的网络有强烈的反感，但这个想法似乎又走了回头路。正如 Yogi Berra 曾指出的那样，这是从头再来似曾相识。然而，Internet 处理路由建立的方式与面向连接网络对路由的处理方式两者之间存在着本质上的区别，所以这种技术肯定不是传统的电路交换。

MPLS 在每个数据包前面增加一个标签，路由器根据数据包标签而不是数据包目标地址实施转发。用标签作为一个内部表的索引，快速查找该表找出正确的输出线路，因而这只是一个表查操作。使用这种技术，路由器的转发速度非常快。这种优势是 MPLS 的最原始设计动机。刚开始时这项独特技术有各种名称，其中包括标记交换（tag switching）。最终，IETF 开始规范化这样的想法，相应的描述发表在 RFC 3031 和许多其他 RFC 文档。随着时间的推移，MPLS 的好处显现了出来，主要表现在灵活的路由和快速以及适合服务质量的转发。

第一个要问的问题是标签放在哪里？由于 IP 数据包并不是针对虚电路设计的，所以在 IP 头并没有可以存放虚电路号的空间。由于这个原因，必须要在 IP 数据包的头前面加上一个新的 MPLS 头。如果从一台路由器到另一台路由器之间的线路使用 PPP 作为成帧协议，那么帧格式中包含了 PPP、MPLS、IP 和 TCP 头，如图 5-62 所示。

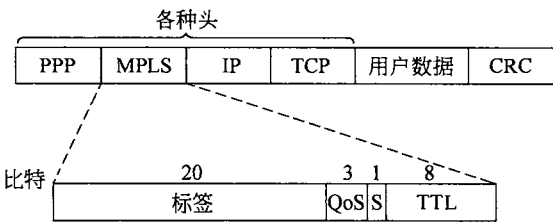


图 5-62 使用 IP、MPLS 和 PPP 传输一个 TCP 段

通用的 MPLS 头有 4 个字节长, 并且包含 4 个字段。其中最重要的是标签 (Label) 字段, 它存放的是索引。QoS 字段指明了服务的类别。S 字段涉及在层次网络中叠加多个标签的做法 (下面会进一步讨论)。TTL 字段指出该数据包还能被转发多少次, 每经过一个路由器被递减 1。如果降为 0, 则该数据包将被丢弃, 这个特性可以防止在路由不稳定的情况下出现无限循环问题。

MPLS 介于 IP 网络层协议和 PPP 链路层协议之间。它不是一个真正的第 3 层协议, 因为它依赖 IP 或其他网络层地址来建立标签路径; 但它也不是一个真正的第 2 层协议, 因为它既可以在多跳之间转发数据包, 也不是一条单一链路。出于这个原因, MPLS 有时称为 2.5 层协议。这是一个例子, 说明真正的协议并不总是符合我们的理想分层协议模型。

MPLS 具有其光明的一面, 因为 MPLS 头既不属于网络层数据包也不属于数据链路层帧的一部分, 所以 MPLS 在相当大的程度上独立于这两层。除此之外, 这个属性还意味着有可能制造出能同时转发 IP 数据包和非 IP 数据包的 MPLS 交换机, 具体如何转发取决于线路上出现的是什么。这个特性就是 MPLS 名字中“多协议”的由来。MPLS 还可以通过非 IP 网络来运载 IP 数据包。

当一个 MPLS 增强型数据包到达一个标签交换路由器 (LSR, Label Switched Router), 标签就被用作查找一个表的索引, 以便确定要使用的出境线路和新标签。所有的虚电路网络几乎都采用这种标签替换技术。标签只有本地意义, 两个不同的路由器可以给两个都到另一台路由器但相互之间毫无关系的数据包分配相同的标签, 这两个数据包的进一步传输将使用同一条出境线路。为了在虚电路的另一端能区别出这两个数据包, 标签必须在每一跳重新进行映射。我们已经在图 5.3 看过这个机制是如何工作的。MPLS 使用了与此相同的技术。

顺便说一句, 有些人将路由器的转发行为和交换行为进行了区分。转发是一个过程, 从一个表中找出与目标地址最佳匹配的表项, 并据此决定将数据包发往哪里。一个例子是 IP 转发使用了最长前缀匹配算法。与此相反, 交换则使用取自数据包的标签作为索引来查询转发表, 这个过程更为简单和迅速。然而, 这些定义还远未普及。

由于大多数主机和路由器都不理解 MPLS, 因此我们还应该考虑何时以及如何把标签附加到数据包上。这事发生在一个 IP 数据包到达 MPLS 网络边缘。标签边缘路由器 (LER, Label Edge Router) 检查数据包的目标 IP 地址和其他字段, 确定该数据包应当遵循哪条 MPLS 路径, 然后右侧的标签贴在数据包前面。进入 MPLS 网络后, 这个标签就被用来转发数据包。在 MPLS 网络的另一边, 标签已经完成自身使命并功成身退; 删除标签后得到原始的 IP 数据包, 然后再被下一个网络转发。这个过程如图 5-63 所示。MPLS 与传统虚电路的一个区别是聚合水平。在通过 MPLS 网络时, 为每个流设置它自己的一组标签当然是可能的; 然而, 对于路由器来说, 更常见的做法是将终止在某个特定路由器或者 LAN 的多个流合并成一组, 并为这些流使用同一个标签。这些被合起来共享同一个标签的流称为属于同一个转发等价类 (FEC, Forwarding Equivalence Class)。该类别不仅覆盖了数据包的去向, 而且覆盖了它们的服务类别 (从区分服务的意义上), 因为从转发的角度而言, 所有的数据包都被同样对待。

若使用传统的虚电路路由方法, 要想把几条具有不同端点 (endpoint) 的独立路径组合到同一个虚电路标识符上是不可能的, 因为在最终接收方无法将它们区分开。有了 MPLS,

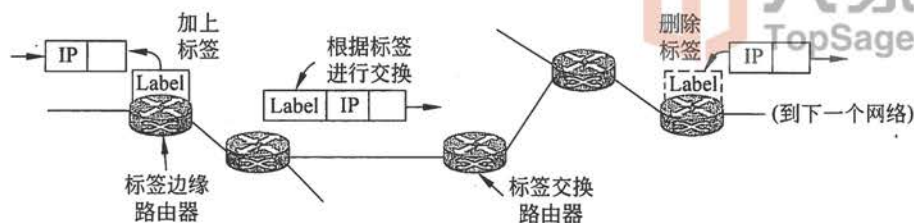


图 5-63 通过一个 MPLS 网络转发 IP 数据包

数据包除了标签以外，仍然包含了它们的最终目标地址，所以，在标签路径的末端，标签头可以被去掉，然后利用网络层的目标地址，按照常规的方法继续向前转发。

其实，MPLS 走得更远。它可以一次在多个标签层次上运行，具体做法是在数据包前面添加多个标签。例如，假设有许多具有不同标签的数据包（因为我们要处理的数据包位于网络中的不同地方），遵循同一条共同的目标路径。对此，不是设置多条标签交换路径，即为每个不同的标签设置一条标签交换路径，而是为这些数据包设置一条单一路径。当已标记的数据包到达这条路径的开始端点处，另一个标签被添加到数据包的前面。这就是所谓的标签栈（stack of label）。最外面的标签指导这些数据包沿着该条公共路径前行。在路径的结束端点处，取出进入标签交换路由前贴上的标签，原来的标签（如果有的话）用来指导数据包进一步的转发。图 5-62 中的 S 标志位告诉删除外层标签的路由器是否还剩下额外的标签。对于最底部的标签，该标志位设为 1；而对所有其他标签，该标志位设置为 0。

最后一个我们会考虑的问题是如何建立标签转发表，因而数据包可遵循该表的指示被转发。这是 MPLS 与传统虚电路设计两者之间的主要区别。在传统的虚电路网络，当用户希望建立一个连接时，会向网络发出一个设置包，通过该包来创建路径并生成转发表的表项。MPLS 在设置阶段不涉及用户。如果用户除了发送数据包之外，还要做任何其他事情将打破现有的 Internet 软件。

相反，转发信息由一些协议设置，这些协议一般是路由协议和连接建立协议的组合。这些控制协议与标签转发分离得干干净净，这种分离具有可以使用多个不同控制协议的便利。其中的一个变种是这样工作的：当路由器启动时，它会检查看看自己是哪些路由的最终目的地（例如，哪个前缀属于它的接口）；然后它为这些路由创建一个或多个 FEC，并且为每个 FEC 分配一个标签，然后把这些标签发给其邻居。反过来，这些邻居在自己的转发表中添加这些标签，再把新标签发送给其邻居；如此一般地往外传播，直到所有的路由器都获得了相应的路径。资源也可以像路径构造一样被预留下来，以便保证适当的服务质量。其他的变种可以设置不同的路径，比如流量工程路径考虑了未使用的容量，并且按需创建路径以便支持服务产品，比如服务质量。

尽管 MPLS 背后的基本思想非常直截了当，但它的细节异常复杂，并且它有许多变种方法和实际部署的使用实例。有关更多的信息，请参考（Davie 和 Farrel，2008；Davie 和 Rekhter，2000）。

5.6.6 OSPF——内部网关路由协议

我们现在已经完成了如何在 Internet 转发数据包的学习。现在继续前进到下一个主题：

Internet 路由。正如我们前面提到的, Internet 由大量的独立网络或自治系统(AS, Autonomous System) 构成, 并由不同的组织运营, 这些组织通常是公司、大学或 ISP。在自己网络内部, 一个组织可以使用自己的内部路由算法, 或者更流行的名称叫域内路由算法(intradomain routing)。不过, 流行的只有极少数几个标准协议。在本节, 我们将了解域内路由问题, 并考查 OSPF 协议, 这是一个被普遍实际使用的路由协议。域内路由协议也称为内部网关协议(interior gateway protocol)。在下一节, 我们将探讨独立运营网络之间的路由问题, 或域间路由(interdomain routing)问题。在这种情况下, 所有网络必须使用相同的域间路由协议和外部网关协议(exterior gateway protocol)。Internet 采用的域间路由协议是边界网关协议(BGP, Border Gateway Protocol)。

早期的域内路由协议采用了距离矢量的设计思想, 基于分布式 Bellman-Ford 算法, 该算法继承自 ARPANET。路由信息协议(RIP, Routing Information Protocol)是当时运行的一个主要例子。在小型网络系统中 RIP 运作良好, 但随着网络规模变得越来越大它工作得就不那么好了; 而且它还遭受无穷计数问题的困扰, 收敛速度一般很慢。因为这些问题, 1979 年 5 月 ARPANET 切换到一个链路状态协议, 1988 年 IETF 开始为域内路由设计一个链路状态路由协议。该协议在 1990 年成为标准, 它就是开放最短路径优先(OSPF, Open Shortest Path First)。它借鉴了另一个称为中间系统到中间系统(IS-IS, Intermediate-System to Intermediate-System)的协议, 该协议已经成为一个 ISO 标准。由于它们的共同根源, 这两个协议非常的大同小异。有关它们的完整故事, 请参见 RFC 2328。它们在域内路由协议中占有绝对的优势, 大多数路由器制造商都同时支持这两个协议。OSPF 更广泛地应用在公司网络, 而 IS-IS 则更多地应用在 ISP 网络。这两者之间, 我们将给出 OSPF 如何工作的大致轮廓。

由于有了其他路由协议的长期工作经验, 负责设计新协议的工作组列出了一系列必须满足的需求。第一, 该算法必须发表在公开的文献中, 这便是 OSPF 中 O(开放的)的含义。由某一家公司拥有的私有方案是无法做到这一点的。第二, 新的协议必须支持多种距离度量, 包括物理距离、延迟等。第三, 它必须是一个动态算法, 能够自动而且快速地适应网络拓扑变化。

第四, 对于 OSPF 来说这是新的需求, 它必须支持基于服务类型的路由。新的协议必须能够区分实时流量和其他的流量, 并使用不同的路由方法。当时, IP 协议有一个服务类型(type of service)字段, 但是没有一个路由协议使用这个字段。OSPF 也包含了该字段, 但是仍然没有人使用它, 最终它又被去掉了。也许这种需求有点超前, 因为它先于 IETF 的区分服务工作展开前提出, 区分服务模式使得服务质量重新焕发了活力。

第五条与上面一条有关, OSPF 必须实现负载均衡, 即把负载分散到多条线路上。大多数以前的协议都将所有的数据包通过最优路径发送出去, 即使存在两条同等程度好的路由也只选择一条使用, 其他路径根本不用。在许多情况下, 将负载分散到多条线路上可以获得更好的性能。

第六, 必须支持层次化系统。到 1988 年, 一些网络已经增长到相当大的规模, 以至于任何一台路由器都不可能知道其完整的拓扑结构。OSPF 必须设计成不要求路由器知道完整的拓扑结构也能很好地工作。

第七, 要求提供适度的安全性, 以防止恶作剧的学生向路由器发送虚假路由信息来欺

骗路由器。

第八，对于那些通过隧道连接到 Internet 的路由器，新协议也必须能够对它们进行处理。以前的协议并不能很好地解决这样的问题。

OSPF 同时支持点到点链路（比如，SONET）和广播网络（例如，大多数局域网）。其实，它能够支持拥有多个路由器的网络，这些路由器中的每一个都可以直接与其他路由器通信（称为多路访问网络），即使它们没有广播能力。此前的协议不能很好地处理这种情况。

图 5-64（a）给出了一个自治系统网络。这里主机被省略了，因为它们在 OSPF 中通常不起作用，真正参与路由协议的是路由器和网络（其中可能包含主机）。图 5-64（a）中的大多数路由器通过点到点链路连接到其他路由器和网络，最终到达这些网络中的主机。然而，路由器 R3、R4 和 R5 则是通过一个广播局域网连接，诸如交换式以太网。

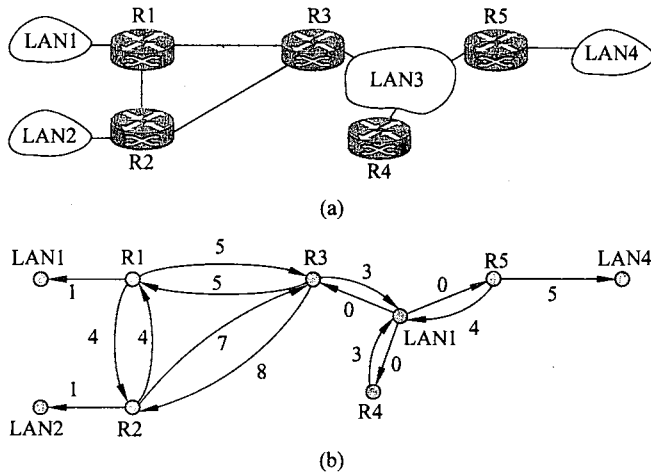


图 5-64
(a) 一个自治系统示例；(b) 针对图 (a) 的图形表示

OSPF 的工作方式本质上是对一张图进行操作：将一组实际网络、路由器和线路抽象到一个有向图中，图中的每条弧有一个权值（距离、延迟等）。两台路由器之间的点到点连接可以用一对弧来表示，每个方向上一个，两个方向上的权值可以不同。广播网络用一个节点表示，加上网络上每台路由器用一个节点表示，从网络节点到路由器节点之间的弧段权值为 0。尽管如此，它们是重要的，因为没有它们就没有通过网络的路径。其他网络只有主机，因此只需要一条到达网络的弧，没有返回弧。这种结构使得路由可以到达主机但不能穿过主机。

图 5-64（b）显示的是图 5-64（a）所示网络的图形表示。OSPF 协议从根本上做了两件事情，首先用一个类似这样的图来表示实际的网络，然后每个路由器使用链路状态方法计算从自身出发到所有其他节点的最短路径。有可能协议会发现多个同样短的路径，在这种情况下，OSPF 记住最短路径集合，并在报文转发期间把流量分摊到这些路径上。这种多路径路由方法有助于负载均衡。该方法称为等价成本多路径（ECMP, Equal Cost MultiPath）。

Internet 中的许多 AS 本身非常庞大，而且不便于管理。OSPF 可以将这样的 AS 划分成编号的区域（area），每个区域是一个网络，或者一组互连的网络。区域不能相互重叠，但

是也不必面面俱到，也就是说有些路由器可能不属于任何一个区域。全部属于一个区域的路由器称为内部路由器（internal router）。区域是单个网络的一种泛化形式。在区域外部，能见到的是它的目的地而不是拓扑结构。

每个 AS 有一个骨干区域（backbone area），称为 0 号区域。该区域中的路由器称为骨干路由器（backbone router）。所有区域都必须连接到骨干区域，连接方式有可能会通过隧道进行；所以，从 AS 内的任何一个区域出发，经过骨干区域，总是有可能到达该 AS 的任何其他区域。在图形表示法中，隧道也用一个弧来表示，并且有一个权值。如同所有其他的区域一样，骨干区域的拓扑结构对于其外部也是不可见的。

每个连接到两个或更多区域的路由器称为区域边界路由器（border router）。它必须是骨干区域的一部分。区域边界路由器的工作任务：概括本区域的目的地信息并注入到与自己连接的其他区域。这种概括包含成本信息，但不包括区域内的所有拓扑细节。传递成本信息可以使得其他区域内的主机找到进入本区域的最好区域边界路由器。不传递拓扑信息可以减少流量和简化其他区域路由器的最短路径计算。然而，如果只有一个边界路由器通往区域外，甚至路由信息概要都不需要传递。通往该区域外部目的地的路由总是被指令“前往边界路由器”，这类区域称为存根区域（stub area）。

最后一种路由器是 AS 边界路由器（AS boundary router）。它把通往其他 AS 的外部路由注入到本区域。然后外部路由就呈现为可以通过 AS 边界路由器可达的目的地，该路由当然会有某种成本。外部路由可以注入一个或更多个 AS 边界路由器。图 5-65 给出了自治系统、区域和各种路由器之间的关系。一个路由器可以扮演多种角色，例如，一个边界路由器同时还是一个骨干路由器。

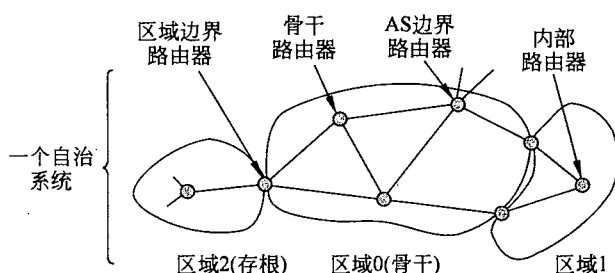


图 5-65 OSPF 中 AS、骨干网和区域相互之间的关系

在正常操作期间，每个区域内的路由器有相同的链路状态数据库，并运行相同的最短路径算法。其主要工作是计算从自身出发到每个其他路由器和整个 AS 内网络的最短路径。区域边界路由器需要所有与之连接区域的数据库，并且为每个区域分别运行最短路径算法。

对于在同一区域内的源和目的地，选择最好的区域内路由（全部位于该地区内）。对于不在同一个区域内的源和目的地，区域间路由必须先从源所在区域到骨干区域，再从骨干区域到目标区域，最后到达目的地。这种算法强制把 OSPF 配置成星型结构，骨干区域相当于集线器，其他区域是向外辐射区域。因为路由算法选择的是最小成本路由，因此位于网络不同位置的路由器可能会选择不同的区域边界路由器进入骨干区域和目标区域。从源到目的地的数据包被“如此这般地”路由，它们不需要封装或者隧道（除非目标区域与骨干区域的唯一连接是一个隧道）。此外，通往外部目的地的路由如果需要可以包括外部成本，

或者仅包含 AS 内部成本。外部成本指从 AS 边界路由器通往目的地的外部路径的成本。

当一台路由器启动时，它在所有的点到点线路上发送 HELLO 消息，并且通过 LAN 将 HELLO 消息组播到一个包含所有其他路由器的组。每台路由器从应答消息中得知谁是自己的邻居。同一个 LAN 上的路由器都是邻居。

OSPF 协议需要在邻接的路由器之间相互交换信息才能工作，邻接（adjacent）路由器与邻居路由器是不同的。尤其是，让一个 LAN 中的每台路由器都跟本 LAN 中的其他每台路由器进行交换路由信息显然非常低效。为了避免出现这样的情形，OSPF 要求从每个 LAN 中选举一台路由器作为指定路由器（designated router）。指定路由器与本 LAN 上的所有其他路由器是邻接的，并且与它们交换信息。实际上，它就是一个代表本 LAN 的单个节点。邻居但不是邻接的路由器相互之间并不交换信息。有一台备份的指定路由器总是保持最新的状态数据，以便缓解主指定路由器崩溃时的转接和取代主指定路由器的需要。

在正常操作过程中，每台路由器周期性地泛洪 LINK STATE UPDATE 消息到它的每台邻接路由器。这些消息给出了它的状态信息，并提供了拓扑数据库用到的成本信息。这些泛洪消息需要被确认，以保证它们的传输可靠性。每条消息都有一个序号，路由器据此判断一条入境 LINK STATE UPDATE 消息比它当前拥有的信息更老还是更新。当一条线路启用、停止或者其成本发生改变时，路由器也要发送 LINK STATE UPDATE 消息。

DATABASE DESCRIPTION 消息给出了由发送方持有的所有链路状态表项的序号。通过把自己相应的值与发送方传过来的这些值进行比较，接收方即可决定谁拥有最新的值。当一条链路启动时使用这些消息。

通过使用 LINK STATE REQUEST 消息，每一对邻接路由器中的任一个路由器都可以向另一个路由器请求链路状态信息。这个算法的结果是，每一对邻接路由器都可检查谁有最新的数据；新的信息就是通过这种方式被传播到整个区域。所有这些消息都是以 IP 数据包的形式直接被发送出去。图 5-66 概括了这 5 类消息。

消息类型	描述
HELLO	用来发现所有的邻居
LINK STATE UPDATE	提供发送者到其邻居的成本
LINK STATE ACK	对链路状态更新消息的确认
DATABASE DESCRIPTION	声明发送者的链路状态更新情况
LINK STATE REQUEST	请求链路状态信息

图 5-66 OSPF 的 5 类消息

最后，我们把所有的工作综合到一起。通过泛洪法，每个路由器把它与其他路由器和网络的链路以及链路成本告诉给它所在区域中的所有其他路由器。这些信息使得每台路由器都可以构建出它所在区域的拓扑图，并且计算最短路径。骨干区域也是这样工作的。而且，为了计算出从每个骨干路由器到每个其他路由器的最佳路由，路由器还要接受来自每个区域边界路由器的信息。最佳路由信息又被传回到区域边界路由器，区域边界路由器再将这些信息在本区域内广播。利用这些信息，内部路由器可以选择通往区域外目的地的最优路由，包括通向骨干区域的最佳出口路由器。

5.6.7 BGP——外部网关路由协议

在一个 AS 内部，推荐使用的路由协议是 OSPF 和 IS-IS。在 AS 之间，则可以使用另一个协议，称为边界网关协议（BGP, Border Gateway Protocol）。之所以在 AS 之间需要一个完全不同的协议，是因为域内协议和域间协议的目标不同。域内协议所需要做的只是尽可能有效地将数据包从源端传送到接收方，它不必考虑政治方面的因素。

相反，域间路由协议则必须要考虑大量和有关政治的因素（Metz, 2001）。例如，一个公司的 AS 可能希望能给所有的 Internet 站点发送数据包，同时也能够接收来自任何一个 Internet 站点的数据包。然而，它可能不愿意承载那些源自一个外部 AS 而终止于另一个外部 AS 的数据包，即使它自己的 AS 正好位于这两个外部 AS 之间的最短路径上（“那是他们的问题，不关我们的事”）。另一方面，它可能愿意转送其邻居们的流量，或者愿意为那些已经付费的一些特殊 AS 提供流量中转服务。例如，电话公司可能很愿意为它们的客户充当运载工具，但是不愿意为别人也提供这样的服务。无论是一般意义上的外部网关协议，还是特殊的 BGP 协议，它们都被设计成允许多种路由策略，这些策略可被强制用在那些跨越 AS 的流量传送上。

典型的路由策略可能涉及政治、安全或者经济方面的考虑因素。下面是一些可能的路由限制例子：

- (1) 教育网络不承载商业流量。
- (2) 五角大楼发出的流量永远不要走经过伊拉克的路径。
- (3) 使用 TeliaSonera 而不用 Verizon，因为前者便宜。
- (4) 不要使用澳大利亚的 AT&T，因为它的性能太差。
- (5) 起止于苹果的流量不应该经过谷歌中转。

正如你从这个列表可能想象的那样，路由策略可以因人而异。它们通常是专有的，因为它们包含了特定的敏感商业信息。然而，我们可以通过描述一些模式来捕捉上述公司的理由，而且常常以此作为出发点。

路由策略的实施决定了哪些流量可以流过 AS 之间的哪些链路。一个常见的政策是客户 ISP 给提供商 ISP 付费，以便将数据包传送到 Internet 上的任何其他目的地以及接收来自 Internet 上任何其他目的地的数据包。可以说客户 ISP 从提供商 ISP 购买了中转服务（transit service）。这就像家庭客户从 ISP 购买了 Internet 接入服务一样。为了能工作，提供商应该把到达 Internet 上全部目的地的路由通过它们之间的链路通告给客户。这样，客户就有一条用来传送数据包到任何地方的可用路由。相反，客户应该向提供商通告它自己网络通往目的地的路由。这样提供商可以只给客户发送要去往来那些目的地址的流量；客户不希望处理来往其他目的地的流量。

我们看一个中转服务例子，如图 5-67 所示。这里有 4 个相互连接的 AS。连接通常采用了 Internet 交换点（IXP, Internet eXchange Point）的链路，为了与其他 ISP 连接，许多 ISP 都有一条链路连到该设施。AS2、AS3 和 AS4 都是 AS1 的客户，它们从 AS1 购买了中转服务。因此，当源 A 给目的地 C 发送时，数据包从 AS2 经过 AS1，最后到达 AS4。路由通告的方向和数据包传送方向相反。为了源端能经过 AS1 到达 C，AS4 向它的中转服

务提供商，即 AS1 通告 C 是一个目的地。以后，AS1 向它的其他客户公告到达 C 的路由，其中包括 AS2，以便客户知道它们可以通过 AS1 发送流量给 C。

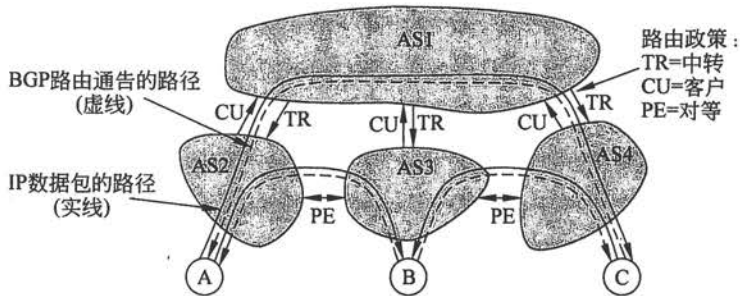


图 5-67 4 个自治系统的路由政策

在图 5-67 中，所有其他 AS 向 AS1 购买中转服务。这种中转服务提供了良好的连接性，使它们可以和 Internet 上的任何主机联系。然而，它们必须为此特权付出代价。假设 AS2 和 AS3 之间有大量的流量需要交流。由于它们的网络早就连接，因此如果它们愿意，它们可以使用不同的政策——直接给彼此发送免费流量。这将减少必须通过 AS1 替它们传递的流量，并有望降低它们的账单。这一政策称为对等（peering）传输。

为了实现对等传输，两个 AS 相互通告到达目的地在自己网络的路由。这样做就有可能使 AS2 把从 A 到 B 的数据包发给 AS3，反之亦然。但是，必须注意，对等不等于中转。在图 5-67 中，AS3 和 AS4 相互对等，这种对等关系允许从 C 到 B 的流量可直接发送到 AS4。如果 C 要发送一个数据包给 A 会发生什么事呢？AS3 仅仅通告了一条到 B 的路径给 AS4，并没有通告有一条到 A 的路由。因此，流量无法从 AS4 传到 AS3，再传到 AS2，即使存在一条物理路径。这种限制正是 AS3 所希望得到的，它与 AS4 对等交换流量，但不希望运载来自 AS4 去往 Internet 其他地方的流量，因为它得不到对方支付的这笔中转费用。相反，AS4 从 AS1 获得中转服务。因此，正是 AS1 负责携带从 C 到 A 的数据包。

现在，我们知道了有关中转流量和对等流量，我们可以看到 A、B、和 C 都有中转安排。例如，A 必须购买 AS2 的 Internet 接入服务。A 可能是一台简单的家庭计算机，或者一个具有多个局域网的公司网络。然而，它并不需要运行 BGP 协议，因为它只是一个存根网络，只有一条链路与 Internet 的其余部分连接。所以，它给网络外部目的地发送数据包的唯一途径就是通往 AS2 的链路，除此之外无路可寻。这条路径的设置非常简单，只需设立一个默认路由即可。出于这个原因，我们没有在图中显示出作为自治系统的 A、B 和 C 参与域间路由。

另一方面，有些公司的网络连接到多个 ISP。这种技术主要用来提高可靠性，因为如果通往一个 ISP 的路径失败，公司可以使用通往其他 ISP 的路径。这种技术称为多穴寻址（multihoming）。在这种情况下，公司的网络有可能运行域间路由协议（比如 BGP），告诉其他 AS 通过哪些 ISP 链路可以到达哪些地址。

这些中转和对等流量的政策可能有许多变种，但它们早已显示了路由通告的商业关系和控制，以及这些因素如何实现不同类型的政策。现在我们要更详细地考虑运行 BGP 的路由器如何相互通告路由信息和选择转发数据包的路由。

BGP 是距离矢量协议的一种形式，但它与域内距离矢量协议（比如 RIP）有很大的不

同。我们已经看到用政策而不是最小距离来选择使用哪些路由。另一个很大的区别是，BGP 不仅维护到每个目的地的成本，而且每个 BGP 路由器还跟踪所使用的路径。这种方法称为路径矢量协议 (path vector protocol)。路径由下一跳路由器 (有可能在 ISP 的另一侧，不一定相邻) 和一系列 AS 或者 AS 路径组成，这些自治系统或者相应的路径序列是该路由必须遵守的 (以相反顺序给出)。最后，一对 BGP 路由器通过建立 TCP 连接而相互通信。这种方式既提供了可靠的通信，也隐藏了正在穿越的网络细节。

图 5-68 中例子显示了 BGP 通告路由的过程。这里有三个自治系统，中间的 ISP 为左右两个 ISP 提供中转服务。AS3 启动一个前缀 C 的路由通告。当该路由通告信息通过跨链路到达 R2c (在图的上方)，它就具有一条到 AS3 的简单路径，并且下一跳路由器是 R3a。在底部，它有一条相同的 AS 路径，但下一跳路由器不同，因为它来自不同的链路。这则路由由广告继续传播，并跨越 AS 边界进入 AS1。在路由器 R1a (在图的上方)，AS 路径为 AS2、AS3，并且下一跳是 R2a。

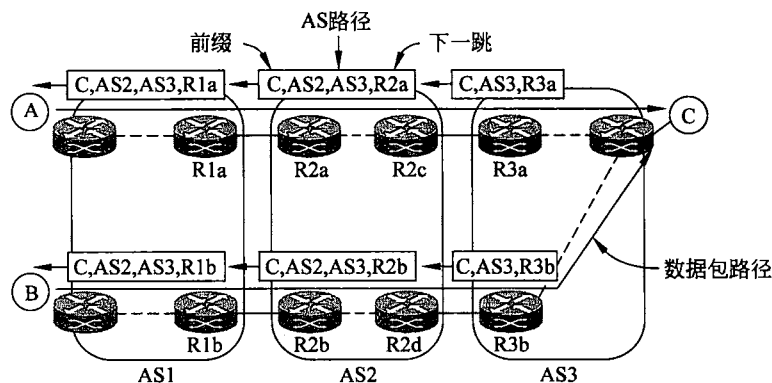


图 5-68 BGP 路由通告的传播

路由通告消息中携带完整的路径易于让接收路由器发现和打破路由循环。相应的规则是每个路由器在往 AS 外部发送路由时附上自己的 AS 编号 (这就是为什么列出的路径是相反顺序的)。当路由器收到一个路由通告时，它会检查消息，检查自己的 AS 号是否已经出现在 AS 路径中。如果是，则说明检测到了一个路由循环，因而必须丢弃路由通告。然而，有点讽刺意义的是，20 世纪 90 年代后期人们才认识到尽管有这种预防措施，BGP 仍然遭受无穷计数问题的困扰 (Labovitz 等, 2001)。BGP 没有长寿的路由环路，但有时路由会收敛得很慢，而且有中转循环。

给出一个 AS 列表来指定一条路径是非常粗糙的表达方式。一个 AS 可能是一个小公司，也可能是一个国际骨干网络。没有任何途径告知这些路由的详情，BGP 甚至不去尝试了解。因为不同的 AS 可能使用不同的域内路由协议，因此这些路由协议的成本无法比较；即使它们能相互比较，一个 AS 或许还不愿意透露其内部的路由度量值。这正是域间路由协议与域内路由协议在方式上的不同所在。

至此，我们已经看到了两个 ISP 如何通过链路发送路由通告消息。我们仍然需要一些方法将 BGP 路由从 ISP 的一侧传播到另一侧，只有这样它们才能被发送到下一个 ISP。这个任务可以由域内协议承担，但由于 BGP 非常善于扩展到大型网络，因此通常使用 BGP 的一个变种。这就是所谓的内部 BGP (iBGP, internal BGP)，以示区别于 BGP 的常规意

用，即外部 BGP（eBGP，external BGP）。

在 ISP 内部传播路由通告有一定的规则：位于 ISP 边界的每个路由器为了一致性，要学习所有其他边界路由器看到的路由。如果 ISP 的一个边界路由器学习到一个去往 IP 128.208.0.0/16 的前缀，那么该 ISP 的所有其他路由器都要学习这个前缀。然后从 ISP 的任何地方都可到达该前缀，不管数据包从其他 AS 如何进入 ISP。

为避免混乱我们并没有在图 5-68 中显示这种传播，但是，例如，路由器 R2b 知道通过顶部的路由器 R2c 或者底部的路由器 R2d 可以到达 C。随着路由在 ISP 内穿越，下一跳得到更新，因此位于 ISP 远端的路由器知道使用另一侧的哪些路由器离开 ISP。这就是为什么最左边的路由看到的下一跳路由器在同一个 ISP 内，而不是下一个 ISP 内的路由器。

现在，我们可以描述 BGP 协议的最关键部分，就是路由器如何为每个目的地选择使用哪条路由。每个 BGP 路由器从与它连接的下一个 ISP 路由器学习如何到达给定目的地的路由，也可以从所有其他边界路由器那里（它们已经从与其他 ISP 连接的路由器上听到不同的路由）学习到给定目标的路由。每个路由器必须决定这些路由集合中哪条路由最好。最终的答案是由 ISP 指定某些政策，根据政策来挑选其中一条首选路由。然而，这种解释太笼统，无法令人满意，所以我们可以至少说明一些共同策略。

第一个策略，优先选择通过对等网络的路由，而不是通过中转提供商的路由。因为前者免费，而后者要付成本费用。类似的策略是给予客户路由最高的优先。只有良好的商业模式才直接给付费客户发送流量。

另一种不同的策略是将“短路径更好”作为默认规则。这个规则值得商榷。因为一个 AS 可能是任何规模的网络，所以通过三个小 AS 的路径实际上可能比通过一个大 AS 的路径短。然而，平均来说短路径趋向于更好，并且这条规则是一个普遍规律。

最后一个策略是优先选择具有 ISP 内最小成本的路由。这就是图 5-68 实施的策略。从 A 发往 C 的数据包从顶部路由器 R1a 离开 AS1。从 B 发出的数据包通过底部路由器 R1b 出口。这么选择的原因在于无论 A 和 B 采取的都是离开 AS1 最小成本路由或者最快路由。因为 A 和 B 位于 ISP 的不同部分，因此对它们每个来说离开 AS 的最快出口是不同的。同样的事情发生在数据包通过 AS2。在最后一站，AS3 必须用自己的网络来运载来自 B 的数据包。

这一策略称为提前退出（early exit）或热土豆路由（hot-potato routing）。它有很好奇的副作用，即使得路由呈现不对称性。例如，考虑 C 发送一个数据包返回到 B 时采取的路径。数据包将通过顶部的路由器很快离开 AS3，以避免浪费资源；同样，当 AS2 以尽可能快的速度将该数据包传给 AS1 时，它一直留在顶部的路由上；然后，该数据包将在 AS1 内经过一条较长的路程。这是一条从 B 到 C 所用路径的镜像。

上述讨论应该明确每个 BGP 路由器从已知的可能路由中选择自己的最佳路由。情况并非如人们天真地预期那样，BGP 按照 AS 粒度来选择路径，OSPF 选择每个 AS 的内部路径。BGP 和内部网关协议的集成相当的根深蒂固。这意味着，例如，BGP 可以找到从一个 ISP 到下一个 ISP 的最好出口点，并且这个点在 ISP 内是各不相同的，如同热土豆策略情况。这也意味着位于一个 AS 内不同部分的 BGP 路由器到达相同的目的地可能选择不同的 AS 路径。ISP 必须小心配置全部的 BGP 路由器，以便自由地做出兼容的路由选择，但是这可以在实践中逐步完成。

令人惊讶的是，我们的讨论只触及了 BGP 的皮毛。有关详细信息，请参阅由 RFC4271 和相关 RFC 文档给出的 BGPv4 规范。然而，要认识到其复杂性更多地在于政策，这不是 BGP 协议规范所描述的内容。

5.6.8 Internet 组播

普通的 IP 通信发生在一个发送方和一个接收方之间。然而，对于有些应用，它们需要一个能够同时向大量接收方发送数据的进程。这样的应用例子有许多，比如给许多观众视频直播体育比赛、向复制服务器池更新程序以及处理数字会议（即多方会议）中的电话呼叫。

IP 用 D 类 IP 地址来支持一对多的通信，或组播。每个 D 类地址标识了一组主机。总共有 28 位可用于标识组播组，因此网络中可同时并存 250 万个组。当一个进程给一个 D 类地址发送数据包时，网络会尽力而为地将这些数据包投递给指定组中的所有成员，但是并不保证一定投递成功。有些成员可能收不到数据包。

IP 地址 224.0.0.0/24 范围内的地址保留用作本地网络组播。在这种情况下，不需要路由协议的支持。带有一个组播地址的数据包被简单广播到局域网，从而达到组播的目的。局域网上的所有主机接收广播数据包，只有属于组成员的主机对该数据包进行处理。路由器不会将数据包转发到局域网外。本地组播地址的例子有：

- 224.0.0.1 LAN 上的全部系统
- 224.0.0.2 LAN 上的全部路由器
- 224.0.0.5 LAN 上的全部 OSPF 路由器
- 224.0.0.251 LAN 上的全部 DNS 服务器

其他组播地址或许有成员分布在不同的网络上。在这种情况下，就需要一个路由协议。但首先组播路由器必须了解哪些主机属于某个组的成员。一个进程要求它的主机加入到某个指定的组中；它也可以要求它的主机离开该组。每台主机跟踪记录当前它的哪些进程属于哪些组。当一台主机上的最后一个进程离开一个组时，该主机就不再属于这个组。大约每分钟一次，每个组播路由器向它所在 LAN 上的所有主机发送一个查询数据包（当然使用本地组播地址 224.0.0.1），要求这些主机报告自己当前属于哪些组。组播路由器可能与标准的路由器在同一台机器上，也可能分属两台机器。每个主机收到查询消息后，返回一个响应包，其中包括了自己感兴趣的所有 D 类地址。这些查询包和应答包使用了一个称为 **Internet 组管理协议**（IGMP, Internet Group Management Protocol）的协议，该协议由 RFC3376 详细描述。

几个组播路由协议中的任何一个都可用于建立组播生成树，该树给出了一条从发送方到组内所有成员的路径。所用的算法是我们在 5.2.8 节中描述的那些。在 AS 内，主要使用的是**协议独立组播协议**（PIM, Protocol Independent Protocol）。PIM 适合几种场合。在密集模式 PIM 中，算法创建了一棵修剪的逆向路径转发树。这棵树比较适合于组成员分布在网络各处的情况，比如数据中心网络把文件分发给多个服务器。在稀疏模式 PIM 中，算法创建的生成树类似于核心树。这种方式比较适合内容提供商向它 IP 网络上的用户组播 TV。这种设计的一种变体，称为**特定源组播 PIM**（Source-Specific Multicast PIM）可用来优化只有一个发送方的情况。最后，当组成员分布在不止一个 AS 时，需要 BGP 或隧道的组播扩展来创建组播路由。

5.6.9 移动 IP

许多 Internet 用户拥有移动计算机，当他们离开家乡甚至在旅途的过程中，他们也希望能够与 Internet 保持连接。不幸的是，IP 的寻址系统使得这样的异地办公说起来容易做起来很难，正如我们马上要描述的那样。当人们开始产生在任何地点都能上网的需求时，IETF 成立了一个工作组来寻求解决方案。该工作组很快制定出了任何一种解决方案都必须达到的一系列目标，其中最主要的有以下几点：

- (1) 每台移动主机必须能在任何地方使用它的家乡 IP 地址。
- (2) 不允许修改固定主机的软件。
- (3) 不允许改动路由器软件 and 各类表。
- (4) 发给移动主机的大多数数据包不应该绕道而行。
- (5) 移动主机在家时不应该有任何开销。

选择的解决方案是我们在 5.2.10 节描述的那样。简单地说，每个允许用户漫游的网点必须创建一个称为家乡代理（home agent）的助手（helper）。当移动主机出现在一个外地网点时，它获得一个外地网点的新 IP 地址（称为转交地址）；然后移动主机通过该转交地址告诉家乡代理自己现在在哪里。当发给该移动主机的数据包到达家乡网点，恰好移动主机不在家外出到了其他地方，那么家乡代理截取该数据包，并隧道给接入到当前转交地址上的移动主机。不管通信对方是谁，移动主机都可以直接发送应答数据包，但仍把家乡地址作为应答数据包的源地址。这个解决方案符合上述所有目标，除了移动主机的数据包必须走弯路这一点外。

既然我们已经覆盖了 Internet 的网络层，现在可以了解该解决方案的更多细节。对移动性支持的需求首先来自于 IP 寻址模式本身。每个 IP 地址包含一个网络号和主机号。例如，考虑一台机器具有 IP 地址 160.80.40.20/16。160.80 给出了该机器所在网络的编号，40.20 是所在网络内的主机号。世界各地的路由器都有路由表说明到达 160.80 网络该用哪条链路。每当到达一个数据包，如果其目标 IP 地址形如 160.80.xxx.yyy，那么路由器就把它从该条链路发送出去。如果突然间，该地址上的机器被搬运到某个遥远网点，寻址该地址的数据包将继续被路由到其家乡局域网（或路由器）。

在这一阶段，有两种选择，但都没有吸引力。第一，我们可以创建一个到更特殊前缀的路由。也就是说，如果一个遥远网点通告了一条到 160.80.40.20/32 的路由，那么发往该目标地址的数据包首先会被发送到适当的地方。这个选项取决于路由器使用的最长匹配前缀算法。然而，我们必须为单个 IP 地址增加一条到一个 IP 前缀的路由。如果每个人带着计算机移动时都要修改全局 IP 路由，那么每个路由器将有数百万的表项/表项，这是个天文数字的成本。显然这种方法在 Internet 上是行不通的。

第二个选择是改变移动主机的 IP 地址。诚然，发送到家乡 IP 地址的数据包将不再被传递，直到所有相关人员、程序和数据库都知晓移动主机的地址变化情况。但移动主机仍然可以在新的位置使用 Internet 浏览网页和运行其他应用程序。这个选项在更高的层次处理移动性，通常发生在用户带着一台笔记本电脑到咖啡店坐着，通过本地无线局域网络使用 Internet。这个选项的缺点是它要中断某些应用程序，因为移动主机来回移动时没有保持

与网络的连接。

顺便说一句，移动性也可以在一个较低层次处理，比如链路层。这通常发生在一台笔记本电脑通过单个 802.11 无线上网。此时移动主机的 IP 地址不会发生改变，因而网络路径保持不变。这正是无线链路提供了对移动性的支持。然而，移动性的程度非常有限。如果笔记本电脑移动得太远，它就必须通过另一个具有不同 IP 地址的网络接入到 Internet。

针对 IPv4 的移动 IP 解决方案由 RFC 3344 给出。它可在现有的 Internet 路由下工作，允许主机移动外出时用自己的 IP 地址保持与网络连接。为了能工作，移动主机必须能够发现自己在移动。这点可以通过 ICMP 路由器通告 (advertisement) 和恳求 (solicitation) 消息做到。移动主机定期监听路由器的通告报文或通过恳求报文来发现最近的路由器。如果该路由器的地址不同于移动主机平常在家用的路由器地址，则它可以断定自己必定在某个外地网络。如果这个路由器地址自上一次修改以来又发生了变化，则说明移动主机已经转移到了另一个外地网络。同样的机制可以使得移动主机发现它们的家乡代理，从而确定自己从外地返回了家乡。

为了在外地网络得到一个转交 IP 地址，移动主机只需简单地使用 DHCP 服务。另外，如果 IPv4 地址短缺，移动主机可以通过外地代理发送和接收数据包，该外地代理早已经有了一个当地网络上的 IP 地址。移动主机用来发现外地代理使用的 ICMP 机制与用来寻找家乡代理使用的机制相同。一旦移动主机获得一个 IP 地址或找到一个外地代理，它就能够利用当地网络给家乡代理发送信息，告知其当前位置。

当移动主机不在家乡网络时，其家乡代理需要某种方式来拦截发给移动主机的数据包。ARP 协议提供了一个方便的机制。要通过以太网给一个 IP 主机发送数据包，路由器需要知道主机的以太网地址。通常的机制是由路由器发送一个 ARP 请求查询报文，例如问“160.80.40.20 的以太网地址是什么？”。当移动主机在家乡网络时，它以自己的 IP 地址与自己的以太网地址回答 ARP 查询。当移动主机外出时，家乡代理用自己的以太网地址来响应此查询。然后路由器把发给 160.80.40.20 的数据包发给家乡代理。回想一下，这就是所谓的 ARP 代理。

为了在移动主机离开或返回家乡网络时快速更新 ARP 映射，采用了另一个 ARP 技术，称为免费 ARP (gratuitous ARP)。基本上，移动主机或家乡代理自己会发送针对该移动主机 IP 地址的 ARP 查询报文，在该查询报文中提供了正确的答案，因此该路由器注意后及时更新其映射。

家乡代理给移动主机转发数据包其实很简单，在家乡代理和连接在转交地址上的移动主机之间建立一个隧道，然后通过该隧道来转发数据包。具体做法是只要用目标地址是转交地址的另一个 IP 头封装该数据包，当封装后的数据包到达转交地址后，外层 IP 头被拆除，提取出里面的数据包交给移动主机。

与许多 Internet 协议一样，麻烦在细节上，并且大多数往往体现在部署时与其他协议的兼容性细节上。这次存在两种并发症。首先，NAT 盒子依赖于经过的 IP 数据包，通过偷窥其 IP 头来查看 TCP 头或 UDP 头。隧道移动 IP 的原始形式没有使用这些头，因此不能与 NAT 盒子一起工作。对此的解决办法是改变封装，使得封装后的数据包包括 UDP 头。

第二个复杂性在于一些 ISP 检查数据包的源 IP 地址，看它们是否匹配路由协议认可的应该出现的地方。这种技术称为入口过滤 (ingress filtering)，它是一种安全措施，路由器

会丢弃那些看似不正确地址的数据包，因为这有可能是恶意的流量。然而，当移动主机在外地网络，它发送到其他 Internet 主机的数据包的源地址不属于所在的外地网络，所以这样的数据包将被外地网络的路由器丢弃。为了解决这个问题，移动主机可以利用转交地址作为隧道的源把数据包返回给家乡代理；从家乡网络，它们再被发送到 Internet，这时的数据包显然出现在正确地方。其代价是更加迂回的路由。

还有另一个我们没有讨论的问题是安全性。当家乡代理接收到一条消息，请求它将所有给 Roberta 的数据包统统转发给某一个 IP 地址时，那么，除非它能够确认这个请求真的是 Roberta 发送的，并且肯定不是有人在模仿它，否则最好不要贸然就答应这个请求。基于密码学的认证协议可用来实现这样的身份验证任务，我们将在第 8 章中学习这样的协议。

IPv6 的移动性协议建立在 IPv4 的基础。上述方案遭到来自三角路由问题的困惑，即发送到移动主机的数据包在遥远的家乡代理那里拐了个弯。在 IPv6 中，针对该问题进行了路由优化，在最初的数据包遵循冤枉路径之后，移动主机和其他 IP 地址可以使用一条两者之间的直接路径。移动 IPv6 由 RFC 3775 定义。

Internet 还定义了另一种移动性。一些飞机上有内置的无线网络，乘客可以使用他们的笔记本电脑连接到 Internet。这架飞机有一个路由器通过无线链路连接到 Internet 的其余部分（难道你指望有线链路？）。所以，现在我们有一个飞行的路由器，这意味着整个网络在移动。网络移动性设计支持这种场景，而笔记本电脑却全然不知飞机在移动。至于它们所关注的这只是另一个网络。当然，某些笔记本电脑可能会使用移动 IP 来保持它们的家乡地址，而它们实际上在飞机上，所以我们有两个层次的移动性。IPv6 的网络移动由 RFC 3963 定义。

5.7 本章总结

网络层向传输层提供服务，它既可以基于虚电路，也可以基于数据报。在这两种情形下，它的主要任务是将数据包从源端路由到接收方。在数据报网络中，路由决策针对每一个数据包而作；在虚电路网络中，路由决策在建立虚电路时做出。

计算机网络用到了许多路由算法。泛洪是最简单的算法，它把数据包发送到所有的路径上。大多数算法寻找一条最短路径，并能自适应网络拓扑的变化。主要的算法是距离矢量算法和链路状态算法。大多数网络实际使用了其中的某一个算法。其他重要的路由话题包括大型网络的层次路由、移动主机的路由、广播路由、组播路由和选播路由。

网络很容易变得拥塞，从而增加了数据包延迟和丢失。网络设计者企图通过一系列手段来避免拥塞，其中包括设计具有足够容量的网络、选择未拥堵的路由、拒绝接受更多的流量、给源端发信号降低速度以及负载脱落。

处理拥塞控制的下一步是努力获得所承诺的服务质量。一些应用程序在乎吞吐量而另一些应用程序却更关心延迟和抖动。提供不同服务质量的方法包括流量整形、路由器上的资源预留以及准入控制。专门被设计用于提供良好服务质量的几种途径包括 IETF 的综合服务（包括 RSVP）和区分服务。

网络在很多方面都有所不同，所以，当多个网络相互连接时，问题就出现了。当不同

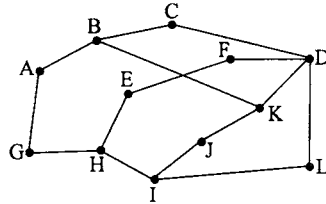
的网络具有不同的最大数据包长度时，可能需要分段。不同网络内部可运行不同的路由协议，但外部必须运行公共的路由协议。有的时候，隧道一个数据包穿越一个敌对网络能解决问题，但是，如果源网络和目标网络的类型不同，这种方法就会失败。

Internet 的网络层有丰富多样的协议。这些协议包括数据报协议 IP 和控制相关协议，比如 ICMP、ARP 和 DHCP。一个称为 MPLS 的面向连接协议携带 IP 数据包穿过某些网络。网络中使用的主要路由协议是 OSPF，穿越网络用的路由协议是 BGP。Internet 正快速消耗着 IP 地址，所以 IPv6 作为 IP 的新版本已经开发出来，并正在被如此之慢地部署着。

习 题

1. 请列举出两个适合使用面向连接服务的计算机应用实例，再列举出两个最好使用无连接服务的计算机应用实例。
2. 数据报网络将每个数据包当作独立的单位进行路由，路由过程彼此独立。虚电路网络不必采用这种方式，因为每个数据包都沿着一条预先确定的路由。试问，这是否意味着虚电路网络不需要具备将单个数据包从任意源端路由到任意接收方的能力呢？请解释你的答案。
3. 请给出 3 个在建立连接时可能需要协商的协议参数例子。
4. 假设所有的路由器和主机都正常工作，并且它们的软件也都没有错误。试问是否存在一个数据包被递交到错误目的地的可能性（无论可能性有多小）？
5. 请给出一个简单的启发式算法，找出一个网络中从指定源端到指定目标端之间的两条路径，要求这两条路径在失去任何一条通信线路的情况下都能够幸免于难（假设存在这样的两条路径）。可以认为路由器足够可靠，因此不必担心路由器崩溃的可能性。
6. 考虑图 5-12 (a) 中的网络。使用距离矢量路由算法，路由器 C 刚刚收到下列矢量：来自 B 的 (5, 0, 8, 12, 6, 2)；来自 D 的 (16, 12, 6, 0, 9, 10)；来自 E 的 (7, 6, 3, 9, 0, 4)。从 C 到 B、D 和 E 的链路成本分别为 6、3 和 5。请给出 C 的新路由表，包括使用的出境线路和成本。
7. 在一个有 50 个路由器的网络中，如果成本以 8 位数字表示，并且距离矢量每分钟交换两次，试问每条（全双工）线路有多少带宽被这个分布式路由算法吞噬掉？假设每个路由器都有三条线路连到其他路由器。
8. 在图 5-13 中，每一行上的两组 ACF 位布尔或 (OR) 的结果是 111。试问这仅仅是一种偶然情况，还是在所有情况下对于所有网络都成立？
9. 一个有 4800 台路由器的网络采用了层次路由。试问对于三层结构来说，应该选择多大的区域和簇才能将路由表的尺寸降低到最小？一个好的起点是假设这样的方案接近最优：有 k 个簇，每个簇有 k 个区域，每个区域有 k 个路由器。这意味着 k 大约是 4800 的立方根（约等于 16）。反复试验找出所有这三个参数在 16 附近的各种组合。
10. 在正文中提到当一台移动主机不在家乡网络时，发送至它本地 LAN 的数据包将被该 LAN 上的家乡代理所截获。针对一个 802.3 LAN 上的 IP 网络，试问家乡代理如何完成这样的截获工作？

11. 参照图 5-6 中的网络。试问若使用以下方法，从 B 发出的一次广播将生成多少个数据包？
 - (a) 逆向路径转发。
 - (b) 汇集树。
12. 考虑图 5-15 (a) 中的网络。想象在 F 和 G 之间加入一条新的线路，但是图 5-15 (b) 中的汇集树仍然不变。试问对于图 5-15 (c) 有什么变化？
13. 请计算下面网络中路由器 C 的组播生成树。组成员分布在路由器 A、B、C、D、E、F、I 和 K 上。



14. 假设图 5-20 中节点 B 刚刚重新启动，它的路由表中没有任何信息。现在它突然需要一条到达 H 的路由，于是它发送广播包，其 TTL 分别被设置为 1、2、3 等。试问它需要经过几轮广播之后才能找到一条路由？
15. 在内部采用虚电路的网络中，可能采用这样一种拥塞控制机制：路由器推迟确认收到的数据包，直到 (1) 它知道沿着虚电路的最后一次传输已经被成功接收，并且 (2) 它有一个空闲缓冲区。为了简单起见，假定路由器使用了停-等式协议，并且每条虚电路的每个方向都有一个专用的缓冲区。如果传输一个数据包（数据或者确认）需要 T 秒，在路径上有 n 台路由器，试问数据包被递交给目标主机的速率是多少？假设几乎没有传输错误，并且从主机到路由器之间连接的速度为无限快。
16. 一个数据报网络允许路由器在必要的时候丢弃数据包。路由器丢弃一个数据包的概率为 p 。请考虑这样的情形：源主机连接到源路由器，源路由器连接到目标路由器，然后目标路由器连接到目标主机。如果任何一台路由器丢掉了一个数据包，则源主机最终会超时，然后再重试发送。如果主机至路由器以及路由器至路由器之间的线路都计为一跳，试问：
 - (a) 每次传输数据包的平均跳数是多少？
 - (b) 数据包的平均传输次数是多少？
 - (c) 每个接收到数据包所需的平均跳数？
17. 针对两个拥塞避免方法 ECN 和 RED，请给出它们之间的两个主要区别。
18. 流量整形采用了令牌桶方案。每 5 微秒一个新的令牌被放入桶中。每个令牌刚好用于一个短数据包，数据包包含 48 个字节数据。试问最大的可持续数据率是多少？
19. 在一个 6 Mbps 网络上有一台主机，其流量通过一个令牌桶进行整形。令牌桶的填充速率为 1 Mbps。初始时令牌桶被填满到容量 8 MB。试问该计算机能以 6 Mbps 的全速率传输多长时间？
20. 图 5-34 中的网络使用 RSVP 预留资源，主机 1 和主机 2 的组播树如图中所示。假设主机 3 请求一条带宽为 2 Mbps 的信道用于接收主机 1 的流，以及一条带宽为 1 Mbps 的信道用于接收主机 2 的流。同时，主机 4 请求一条带宽为 2 Mbps 的信道用于接收主机

- 1 的流；主机 5 请求一条带宽为 1 Mbps 的信道用于接收主机 2 的流。试问在路由器 A、B、C、E、H、K、J 和 L 上，总共需要为这些请求预留多少带宽？
21. 一个路由器可以每秒钟处理 200 万个数据包。提供给路由器的负载为每秒钟 150 万个数据包。如果从源端到接收方的路径上有 10 个路由器，试问路由器花在排队和服务上的时间为多少？
 22. 假设网络采用区分服务模型。考虑使用加速转发服务的用户。试问是否可以保证加速型数据包比常规数据包的延迟更短？为什么是，或者为什么不是？
 23. 假设主机 A 与路由器 R1 连接，R1 又与另一个路由器 R2 连接，R2 与主机 B 连接。假定一个要发给主机 B 的 TCP 消息被传递给主机 A 的 IP 代码，其中包含了 900 个字节的数据和 20 个字节的 TCP 头。请写出在三条链路上传输的每个数据包中 IP 头部的 Total length、Identification、DF、MF 和 Fragment offset 字段。假定链路 A-R1 链路可以支持的最大帧长为 1024 字节，其中包括 14 字节的帧头；链路 R1-R2 可以支持的最大帧长为 512 字节，其中包括 8 字节的帧头；链路 R2-B 可以支持的最大帧长为 512 字节，其中包括 12 字节的帧头。
 24. 一个路由器往外发送大量的 IP 数据包，这些数据包的总长度（数据+头）为 1024 字节。假设这些数据包的生存期为 10 秒，试问路由器运行的最大线速度达到多少才不会发生 IP 数据包的 ID 编号空间重绕的危险？
 25. 一个 IP 数据报使用了严格源路由（Strict source routing）选项，现在它必须被分段。你认为该选项应该被复制到每个段中，还是只需放到第一个段中就足够了？请解释你的答案。
 26. 假定最初的时候 B 类地址的网络部分不是 16 位，而是 20 位。试问将有多少个 B 类网络？
 27. 一个 IP 地址的十六进制表示为 C22F1582，请将它转换成点分十进制表示法。
 28. Internet 上一个网络的子网掩码为 255.255.240.0。试问它最多能够容纳多少台主机？
 29. 尽管 IP 地址特定于一个网络，以太网地址却不是。你能想到一个好理由，说明为什么以太网地址做不到吗？
 30. 从 198.16.0.0 开始有大量连续的 IP 地址可以使用。假设 4 个组织 A、B、C 和 D 按照顺序依次申请 4000、2000、4000 和 8000 个地址。对于每一个申请，请用 w.x.y.z/s 的形式写出所分配的第一个 IP 地址、最后一个 IP 地址以及掩码。
 31. 一个路由器刚刚接收到以下新的 IP 地址：57.6.96.0/21、57.6.104.0/21、57.6.112.0/21 和 57.6.120.0/21。如果所有这些地址都使用同一条出境线路，试问它们可以被聚合吗？如果可以，它们被聚合到哪个地址上？如果不可以，请问为什么？
 32. 从 29.18.0.0 到 29.18.128.255 的一组 IP 地址已经被聚合到 29.18.0.0/17。然而，这里有一个空闲地址块，即从 29.18.60.0 到 29.18.63.255 之间的 1024 个地址还没有被分配。现在这块空闲地址突然要被分配给一台使用不同出境线路的主机。试问是否有必要将聚合地址分割成几块，然后把新的地址块加入到路由表中，再来看是否可以重新聚合？如果没有必要这样做，请问该怎么办呢？
 33. 一个路由器的路由表中有如下的表项：

地址/掩码	下一跳
135.46.56.0/22	Interface 0
135.46.60.0/22	Interface 1
192.53.40.0/23	Router 1
default	Router 2

对于下列 IP 地址，如果到达的数据包带有这些地址，试问路由器该如何处理？

- (a) 135.46.63.10
- (b) 135.46.57.14
- (c) 135.46.52.2
- (d) 192.53.40.7
- (e) 192.53.56.7

34. 许多公司采取这样的策略：通过两个或者多个路由器将公司连接到 Internet。这种冗余度保证了其中一个路由器停机时网络还能使用。试问采用 NAT 策略之后，仍然能正常工作吗？请解释你的答案。
35. 你刚刚向一个朋友解释了 ARP 协议。当你解释完之后，他说：“我明白了，ARP 给网络层提供了一项服务，所以它是数据链路层的一部分。”你该如何向他解释呢？
36. 请给出一种在目标主机上重组 IP 分段的方法。
37. 大多数 IP 数据报重组算法有一个计时器，以免丢失的段永远占用重组缓冲区。假设一个数据报被分为 4 个段。前 3 段到达目的地，但最后一个被延迟了。最终该计时器超时，接收方内存中的 3 个段被丢弃。过了一会儿，最后一段姗姗来迟。试问应该用它做什么呢？
38. 在 IP 中，校验和仅仅覆盖了头，而没有包括数据部分。你认为选择这种设计方案的理由是什么？
39. 有一个人生活在 Boston，现在她带着自己的笔记本电脑去 Minneapolis 旅游。让她惊讶的是，在 Minneapolis 目的地的局域网是一个无线 IP 局域网，所以她根本不需要插网线。试问，她是否仍然需要通过家乡代理和外部代理这一整套过程才能正确地接收电子邮件或者其他流量？
40. IPv6 使用 16 个字节的地址。如果每隔 1 ps 就分配掉一百万个地址，试问整个地址空间可以持续分配多久？
41. IPv4 头中的 Protocol 字段并没有出现在 IPv6 的固定头中。试问为什么？
42. 当 IPv6 协议被引入时，ARP 协议需要作相应的改变吗？如果需要，这种改变是概念性的还是技术性的？
43. 编写一个程序来模拟泛洪路由算法。每个数据包应该包含一个计数器，在每一跳上该计数器值减一。当计数器值到达 0 时，该数据包就被丢弃。时间是离散的，每条线路在每个间隔中只处理一个数据包。请完成该程序的三个版本：所有的线路都被泛洪、除了入境线路外其他所有的线路都被泛洪、只有最佳的 k 条线路（静态选择）才被泛洪。按照延迟和所用带宽比较泛洪算法和确定性路由算法（k=1）的性能。
44. 编写一个程序来模拟使用离散时间的计算机网络。在每个时间间隔中，每个路由器队列中的第一个数据包向前走一跳。每个路由器只有有限数量的缓冲区。如果一个数据

包到来时，路由器没有可用的缓冲区空间，那么该数据包将被丢弃，并且不再重传。同时，另有一个端到端协议，它完全支持超时和确认数据包。最终源路由器会重新生成丢弃的数据包。请给出网络吞吐量与端到端超时间隔之间的函数关系，错误率是一个参数。

45. 编写一个函数来完成 IP 路由器的转发过程。该过程有一个参数，即 IP 地址。它还要访问一张全局表，表项由三元组构成。每个三元组包含三个整数：IP 地址、子网掩码和所用的输出线路。该函数利用 CIDR 技术查询该表，然后返回作为参数给出的 IP 地址所对应的输出线路值。
46. 使用 `traceroute` (UNIX) 和 `tracert` (Windows) 程序跟踪从你的计算机到其他国家不同大学的路由。列出你发现的跨洋链路。可以尝试的一些站点是：

<code>www.berkeley.edu</code>	(California)
<code>www.mit.edu</code>	(Massachusetts)
<code>www.vu.nl</code>	(Amsterdam)
<code>www.ucl.ac.uk</code>	(London)
<code>www.usyd.edu.au</code>	(Sydney)
<code>www.u-tokyo.ac.jp</code>	(Tokyo)
<code>www.uct.ac.za</code>	(Cape Town)

第6章 传 输 层

传输层与网络层一起构成了网络协议层次的核心。网络层使用数据报或虚电路技术为端到端通信提供了数据包交付服务。传输层架构在网络层提供的服务之上，把数据传递服务从两台计算机之间扩展到了两台计算机上的进程之间，并且服务所需的可靠性程度独立于当前使用的物理网络。传输层为应用层使用网络提供了抽象的模式。如果没有传输层，分层协议的整个概念将毫无意义。在本章，我们将详细学习传输层，包括它的服务和 API 设计的选择，其中涉及可靠性、连接和拥塞控制，协议（比如 TCP 和 UDP）和性能等问题的解决。

6.1 传 输 服 务

在下面的章节中，我们将简要介绍传输层服务。我们要考查传输层向应用层提供什么样的服务。为了使传输服务的问题更具体，我们将考查两套传输层原语。先看一个简单的（假想的）原语，以便了解传输层的基本性质，然后再看 Internet 的常用接口。

6.1.1 提供给上层的服务

传输层的最终目标是向它的用户提供高效的、可靠的和成本有效的数据传输服务，它的用户通常是应用层的进程。为了实现这个目标，传输层需要充分利用网络层提供给它的服务。在传输层内，完成这项工作的硬件和/或软件称为传输实体（transport entity）。传输实体可以实现在主机的不同位置，可能在操作系统内核，或者以一个链接库的形式绑定到网络应用中，或者以一个独立的进程运行，甚至可以实现在网络接口卡上。前两种实现方式在 Internet 上最常见。网络层、传输层和应用层之间的（逻辑）关系如图 6-1 所示。

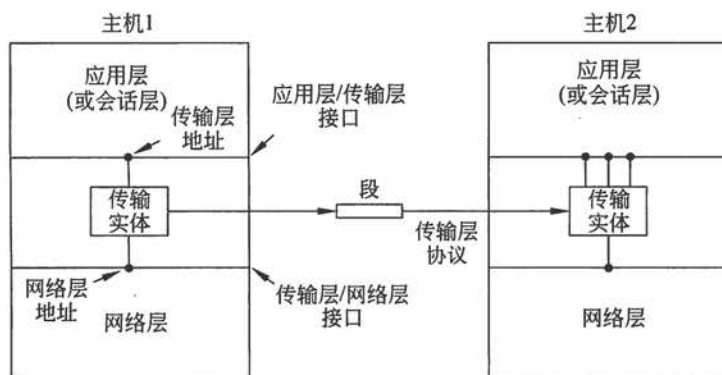


图 6-1 网络层、传输层和应用层

与网络层提供面向连接和无连接两种服务一样，传输层的服务类型也分为两种。面向

连接的传输服务在许多方面与面向连接的网络服务类似，两者的连接都要经历 3 个阶段：连接建立、数据传输和连接释放。在这两层上，寻址和流量控制非常相像。另外，无连接的传输服务与无连接的网络服务也极为相似。然而请注意，在一个面向连接的网络服务之上提供无连接传输服务可能很困难，因为为了发送单个数据包要建立一个连接、发送完毕后还要立即拆除这个连接，效率实在是太低了。

于是，一个很显然的问题出现了：既然传输层服务与网络层服务如此相似，为什么还要设立两个独立的层？为什么一层不够？问题的答案有点微妙，但非常关键。传输层的代码完全运行在用户的机器上，但是网络层代码主要运行在由运营商操作的路由器上（至少对于广域网是如此）。如果网络层提供的服务不够用，怎么办？如果它频繁地丢失数据包该怎么办？如果路由器时常崩溃又该怎么办？

问题发生了，这就是答案。用户对网络层没有真正的控制权，因为他们不拥有路由器，所以不能用更好的路由器或者在数据链路层上用更好的错误处理机制来解决服务太差的问题。唯一的可能是在网络层之上再加一层，由该层来提高网络的服务质量。如果在一个无连接网络中，数据包被丢失或者发生错位，则传输实体可以检测到问题所在，并通过重来弥补这种错误。如果在一个面向连接网络中，传输实体在执行一个漫长传输任务期间，突然接到通知说它的网络层连接已经被意外终止，而且也不知道当前正在传输的那些数据到底怎么样，那么，该传输实体可以与远程的传输实体建立一条新的网络层连接。利用新建立的连接，它可以向对等实体询问哪些数据已经到达，哪些数据还没有到达，然后从中断的地方开始继续向对方发送数据。

本质上，由于传输层的存在，使得传输服务有可能比网络服务更加可靠。而且，传输服务原语可以通过调用库程序来实现，从而使得这些原语独立于网络服务原语。不同网络上的网络服务原语可能有很大的差别（比如，无连接以太网服务可能完全不同于面向连接的 WiMAX 服务）。将网络服务隐藏在一组传输服务原语的背后，带来的好处是，一旦改变了网络服务，只需要替换一组库程序即可；新的库程序使用了不同的底层网络服务，但是实现了同样的传输服务原语。

值得庆幸的是，正是有了传输层，应用程序员才可以按照一组标准的原语来编写代码，并且程序可以运行在各种各样的网络上；他们根本无须处理不同的网络接口，也不用担心传输的可靠性。如果所有实际的网络都完美无缺，具有相同的服务原语，并保证不会发生变化，那么传输层或许就不再需要。然而，在现实世界中，传输层承担了把上层与技术、设计和各种缺陷隔离的关键作用。

基于这个原因，许多人习惯于将网络分成两部分：第 1 层至第 4 层为一部分，第 4 层之上为另一部分。下面的 4 层可以看作是**传输服务的提供者**（transport service provider），而上面的层次则视为**传输服务的用户**（transport service user）。这种服务提供者与服务用户的区分对于协议层的设计有重要的影响，同时也把传输层放到了一个关键位置，因为它构成了可靠数据传输服务的提供者和用户两者之间的主要边界。这就是应用层能看到的传输层。

6.1.2 传输服务原语

为了允许用户访问传输服务，传输层必须为应用程序提供一些操作，也就是说，提供

一个传输服务接口。每个传输服务都有它自己的接口。在本小节中，我们首先介绍一个简单的（假想的）传输服务以及相应的接口，通过它了解传输服务的本质所在。在下一小节我们将介绍一个实际的例子。

传输服务类似于网络服务，但是两者之间有一些重要的区别。最主要的区别在于网络服务毫不掩盖地按照实际网络提供的服务来建立模型。实际网络可能会丢失数据包，所以网络服务一般来说是不可靠的。

与此相反，面向连接的传输服务是可靠的。当然，实际网络并非没有错误，但是，这恰好是传输层的目标——在不可靠的网络之上提供可靠的服务。

作为一个例子，请考虑一个 UNIX 系统中通过管道（或者任何其他进程之间的通信设施）连接起来的两个进程。假设两者之间的连接是十分完美。它们并不想了解有关确认、数据包丢失、拥塞以及诸如此类的情况。它们所要的是一个 100%可靠的连接。进程 A 把数据放进管道的一端，进程 B 可以从管道的另一端将数据取出来。这就是面向连接传输服务的真正含义所在——将网络服务的各种缺陷隐藏起来，因此用户进程只要假设它们两者之间存在一个无错的比特流，即使双方位于不同的机器上时也一样。

另一方面，传输层也可以提供不可靠（数据报）服务。然而，相对来说关于这种服务除了“它是数据报”外并没有太多内容可说，所以本章我们将注意力主要集中在面向连接的传输服务上。不过，有一些应用建立在无连接传输服务上，比如客户机-服务器计算和流式多媒体应用，所以，我们在后面还是会稍微介绍这种服务。

网络服务和传输服务之间的第二个区别在于它们的服务对象不同。网络服务仅仅被传输实体使用。通常用户不会编写自己的传输实体，因此，很少有用户或者程序能看到裸露的网络服务。相反，许多程序（和程序员）可以看到传输原语。因此，传输服务的使用必须非常方便、容易。

为了了解传输服务的基本面貌，请考虑图 6-2 中列出的 5 个原语。这个传输接口是真正的赤裸裸，但它给出了一个面向连接的传输接口应该完成的基本工作。它允许应用程序建立并使用连接，用完之后再释放连接，对于许多应用来说这已经足够了。

原语	发出的包	含义
LISTEN	(无)	阻塞，直到某个进程试图与之连接
CONNECT	CONNECTION REQ	主动尝试建立一个连接
SEND	DATA	发送信息
RECEIVE	(无)	阻塞，直到到达一个 DATA 包
DISCONNECT	DISCONNECTION REQ	请求释放连接

图 6-2 一个简单传输服务的原语

为了看清楚这些原语的可能用法，请考虑一个应用，它有一个服务器和多个远程客户。首先，服务器执行 LISTEN 原语，一般的做法是调用一个库过程，由它执行阻塞该服务器的系统调用，直到有客户请求连接。当一个客户希望与该服务器进行通话时，它就执行 CONNECT 原语。传输实体执行该原语并阻塞调用方，然后给服务器发送一个包。封装在该包有效载荷中的是一条发送给服务器传输实体的传输层消息。

现在我们快速说明术语的用法。由于缺乏更好的表达，我们将使用术语段（segment）来表示传输实体之间发送的消息。一些老协议使用了更加笨拙的名称——传输协议数据单

元 (TPDU, Transport Protocol Data Unit)。这个术语现在已不再使用, 但仍可在一些老文章和书籍中看到该用法。

因此, 段 (传输层之间交换的单元) 被包裹在数据包 (网络层之间交换的单元) 中, 而数据包则被包含在帧 (数据链路层之间交换的单元) 中。当一帧到达时, 数据链路层对帧头进行处理, 如果帧目标地址与本地传递地址匹配, 则把帧的有效载荷字段中的内容传递给网络实体。网络实体对数据包头进行类似处理, 然后把数据包的有效载荷字段内容向上传递给传输实体。这种嵌套关系如图 6-3 所示。

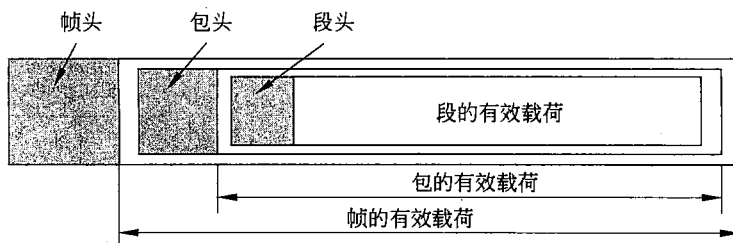


图 6-3 段、包和帧的嵌套关系。

再回到我们的客户机-服务器例子, 客户的 `CONNECT` 调用导致传输实体发送一个 `CONNECTION REQUEST` 段给服务器。当该段到达服务器时, 传输实体检查服务器是否阻塞在 `LISTEN` 状态 (即服务器对处理请求感兴趣)。如果是, 则解除服务器的阻塞, 并给客户送回一个 `CONNECTION ACCEPTED` 段。当该段返回到客户机时, 客户机的阻塞也被解除, 于是连接被建立了起来。

现在双方可以通过 `SEND` 和 `RECEIVE` 原语交换数据。在最简单的形式中, 任何一方都可以执行 (阻塞的) `RECEIVE` 原语, 等待另一方执行 `SEND` 原语。当段到来时, 接收端被解除阻塞; 然后它可以对这个段进行处理, 并发送一个应答。只要双方保持应该谁发送数据的次序, 这种方案就可以工作得很好。

请注意, 在传输层上, 即使一个非常简单的单向数据交换过程也比网络层的交换过程复杂得多。发送的每个数据包 (最终) 都要被确认。携带控制段的数据包也要被确认, 无论是隐式方式还是显式方式。这些确认由使用网络层协议的传输实体来管理, 并且它们对于传输用户是不可见的。类似地, 传输实体需要关心计时器和重传。这些机制对于传输用户全部是透明的。对传输用户而言, 连接就是一个可靠的比特管道: 一个用户在管道一端将比特塞进去, 这些比特就会神奇地出现在管道的另一端。正是这种隐藏复杂性的能力才使得分层协议成为如此强大的工具。

当不再需要一个连接时必须将它释放, 以便释放两个传输实体内部的表空间。中断连接有两种方式: 非对称的和对称的。在非对称方式中, 任何一方都可以发出 `DISCONNECT` 原语, 从而驱使它的传输实体将一个 `DISCONNECT` 段发送给远程的传输实体。当该段到达另一端时, 连接就被释放了。

在对称方式中, 连接的两个方向彼此独立, 因此需要单独关闭每个方向。当一方执行了 `DISCONNECT`, 这意味着它没有更多数据需要发送, 但是它仍然愿意接受对方发送过来的数据。在这种模型下, 只有当双方都执行了 `DISCONNECT` 原语, 一个连接才算真正被释放。

图 6-4 给出了使用这些简单原语建立和释放连接的状态图。每个状态的迁移都是由某种事件触发的，这些事件可能是本地的传输用户执行了一个原语，或者是接收到了一个数据包。为了简化起见，我们假设每个段单独确认；还假设采用了对称的连接释放模型，并且由客户先释放连接。请注意，这种模型相当不成熟，后面我们在描述 TCP 如何工作时会考察更实际的模型。

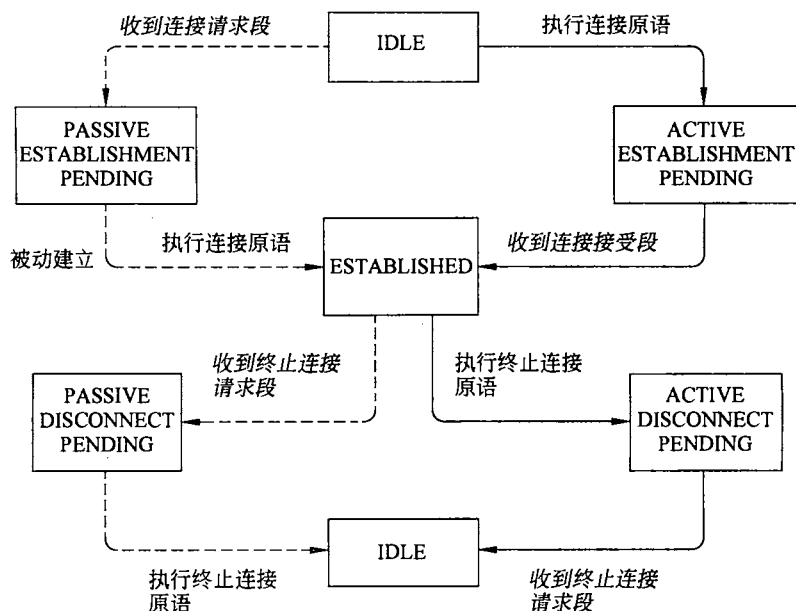


图 6-4 一个简单连接管理方案的状态图
斜体标记的状态转移是由到达的包引起的。实线表示客户的状态序列，虚线表示服务器的状态序列

6.1.3 Berkeley 套接字

现在让我们简要地考查另一组传输原语，即 TCP 所用的套接字（socket）原语。作为 Berkeley UNIX4.2 BSD 软件的一部分。套接字首次发布在 1983 年，这些原语很快得到流行，现在已被许多操作系统广泛应用于 Internet 程序设计中，尤其是基于 UNIX 的系统，Windows 系统也有一个套接字风格的 API，称为“winsock”。

图 6-5 列出了一些原语。粗略地说，这些原语遵循我们第一个例子的模型，但提供了更多的功能和灵活性。我们在这里不去探究相应的段，把讨论留到后面。

原语	含义
SOCKET	创建一个新通信端点
BIND	将套接字与一个本地地址关联
LISTEN	声明愿意接受连接；给出队列长度
ACCEPT	被动创建一个入境连接
CONNECT	主动创建一个连接
SEND	通过连接发送一些数据
RECEIVE	从连接上接收一些数据
CLOSE	释放连接

图 6-5 TCP 的套接字原语